uc3m | Universidad Carlos III de Madrid

Grado en Ingeniería en Tecnologías de Telecomunicación
2025 - 2026

Trabajo Fin de Grado

"Orquestación de Funciones de Red 5G como CNFs en Kubernetes"

Daniel Carballo Reyes

TUTOR

Daniel Diaz Sánchez

Escuela Politécnica Superior de Leganés

Septiembre 2026



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento – No Comercial – Sin Obra Derivada**

Resumen

Este Trabajo de Fin de Grado aborda el despliegue de una red 5G Standalone (SA) completa, incluyendo el núcleo de red (5G Core) y la red de acceso radio (RAN), sobre un clúster de Kubernetes de un único nodo, empleando los componentes software de código abierto de OpenAirInterface (OAI) distribuidos mediante Helm.

El despliegue simula la segmentación de interfaces de red propia del núcleo 5G, separando los planos N2, N3, N4 y N6, mediante Multus CNI y el plugin macvlan; mediante el servicio rfsimulator de OAI se valida el comportamiento funcional del clúster sin necesitar hacer uso de un SDR.

A lo largo del trabajo se documentan y resuelven diversas incidencias surgidas durante el despliegue (conflictos de direccionamiento IP en las interfaces macvlan, enlazado incorrecto de sockets SCTP, registro del gNB frente al AMF y saturación recurrente del buffer de envío del socket TCP que sustituye al enlace radio), empleando una metodología de depuración sistemática por capas apoyada en la inspección de bajo nivel del sistema operativo. El despliegue alcanza conectividad de extremo a extremo: el equipo de usuario software (nr-UE) completa la sincronización de capa física, el registro NAS y el establecimiento de una sesión PDU, obteniendo direccionamiento IP sobre la interfaz túnel oaitun_ue1 y cursando tráfico de plano de usuario contra el UPF a través de la cadena completa radio–GTP-U–N3.

El documento incluye un marco teórico detallado sobre la arquitectura 5G SA, las tecnologías de contenedores y orquestación (Docker, Kubernetes, Helm), las redes multi-interfaz en Kubernetes (Multus, macvlan) y la arquitectura O-RAN, con el objetivo de facilitar la comprensión de la memoria.

Palabras clave: 5G Standalone, OpenAirInterface, Kubernetes, Multus CNI, contenedores, núcleo de red 5G, gNB, rfsimulator.

Índice de contenidos

‘

Resumen 2
Índice de contenidos 3
Índice de figuras 6
Índice de tablas 8
Glosario de acrónimos 9
1. Introducción 12
1.1. Motivación 12
1.2. Objetivos 12
1.2.1. Objetivo general 12
1.2.2. Objetivos específicos 13
1.3. Alcance 13
1.4. Estructura del documento 14
2. Marco teórico 15
2.1. Introducción a las redes móviles 5G 15
2.1.1. Conceptos previos: cliente-servidor y transporte de datos 16
2.2. Arquitectura de la red 5G Standalone 16
2.2.1. El núcleo de red 5G (5GC) 16
2.2.2. La red de acceso radio de nueva generación (NG-RAN) 19
2.2.3. Interfaces y flujo de registro del UE 22
2.2.4. Interfaces y protocolos de comunicación 23
2.2.5. SCTP como transporte de la señalización N2 24
2.2.6. O-RAN: hacia una RAN abierta y desagregada 24
2.3. Virtualización de funciones de red y arquitecturas cloud-native 26
2.3.1. De las máquinas virtuales a los contenedores 26
2.3.2. Adopción cloud-native en el núcleo de red 5G 27
2.4. Kubernetes como plataforma de orquestación de contenedores 27
2.4.1. Arquitectura general de Kubernetes 27
2.4.2. kubectl: interfaz de línea de comandos 29
2.4.3. Minikube: distribución de Kubernetes de un único nodo 30
2.4.4. Helm como gestor de paquetes 30
2.5. Redes avanzadas en Kubernetes: Multus CNI 30
2.6. El proyecto OpenAirInterface (OAI) 32
2.6.1. OAI 5G Core (oai-cn5g) 32
2.6.2. OAI RAN: gNB y nr-UE 32

2.7. Trabajos relacionados 32
3. Diseño e implementación del despliegue 34
3.1. Entorno de trabajo 34
3.1.1. De un despliegue en máquina física con USRP a un entorno virtualizado con rfsimulator 34
3.2. Arquitectura de red del despliegue 35
3.3. Resolución de incidencias en la fase de despliegue del núcleo 36
3.3.1. Conflicto de direcciones IP estáticas duplicadas 36
3.3.2. Enlazado del socket SCTP del AMF a la interfaz macvlan 36
3.3.3. Registro del gNB frente al AMF 37
3.4. Configuración del enlace radio simulado (rfsimulator) 38
3.5. Registro del UE y establecimiento de la sesión PDU 40
3.5.1. Configuración de tiempo real del sistema operativo 42
3.6. Metodología de depuración empleada 43
3.7. Guía de despliegue paso a paso 43
3.7.1. Preparación de la máquina virtual Debian 43
3.7.2. Puesta en marcha de Minikube y Multus CNI 45
3.7.3. Configuración de los values.yaml del 5GC y la RAN 45
3.7.4. Despliegue con Helm/kubectl y verificación de ConfigMaps 47
4. Marco regulador 50
4.1. Estándares técnicos aplicables 50
4.2. Regulación del espectro radioeléctrico 51
4.3. Seguridad de redes 5G 52
4.4. Licencias de software y propiedad intelectual 52
4.5. Protección de datos y privacidad del suscriptor 53
5. Entorno socioeconómico 54
5.1. Presupuesto del proyecto 54
5.2. Impacto socioeconómico 55
5.3. Contribución a los Objetivos de Desarrollo Sostenible 56
6. Conclusiones 57
6.1. Resumen del trabajo realizado 57
6.2. Evaluación de los objetivos 57
6.3. Líneas futuras de trabajo 58
Referencias 60
Especificaciones 3GPP y ETSI 60
Kubernetes, Helm y redes de contenedores 61
OpenAirInterface 62
Trabajos relacionados 62

O-RAN 62
Industria, adopción y presupuesto 63
Marco regulador 63
Anexo 1 – Configuración del Cluster 65
Interfaces de los Pods levantados: 65
Puertos relevantes para rfsim 67
Estado de los Pods 67
Estado de los Deployments 67
Estado de los Persistent Volume Claims (PVC) 67
Ejemplo de Logs NR-UE funcional 68
Ejemplo de Logs gNB funcional 68
Ejemplo de Logs AMF funcional 68
Ejemplo de Logs AUSF funcional 69
Ejemplo de Logs NRF funcional 69
Ejemplo de Logs SMF funcional 69
Anexo 2 – DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE GRADO (TFG) 71

Índice de figuras

Figura 2.1. Comparativa de los modos de despliegue 5G No Independiente (NSA) e Independiente (SA).

Figura 2.2. Arquitectura general de una red 5G Standalone: UE, NG-RAN y 5GC.

Figura 2.3. Funciones de red del 5GC: arquitectura basada en servicios (SBA).

Figura 2.4. Procedimiento de autenticación 5G-AKA.

Figura 2.5. Pila de protocolos de la interfaz NR y alcance del rfsimulator.

Figura 2.6. Arquitectura O-RAN: desagregación del gNB y RAN Intelligent Controllers.

Figura 2.7. Arquitectura de una máquina virtual.

Figura 2.8. Arquitectura de un contenedor.

Figura 2.9. Arquitectura general de Kubernetes (clúster de un único nodo).

Figura 2.10. Pod multi-homed mediante Multus CNI y plugin macvlan.

Figura 3.1. Interfaces de red del Pod oai-amf: eth0, lo y n2 (macvlan).

Figura 3.2. Definición del NetworkAttachmentDefinition oai-amf-n2.

Figura 3.3. Fragmento del values.yaml del chart oai-amf con la IP estática del NAD de N2.

Figura 3.4. Fragmento del ConfigMap del AMF: interfaz n2 y puerto SCTP 38412.

Figura 3.5. Servicio ClusterIP del AMF e interfaces del Pod.

Figura 3.6. Fragmento del values.yaml del chart oai-gnb con el parámetro amfhost.

Figura 3.7. Estado de las conexiones TCP en el Pod del gNB (/proc/net/tcp).

Figura 3.8. Traza del SMF durante el establecimiento de la sesión PDU.

Figura 3.9. Interfaz túnel oaitun_ue1 operativa en el Pod del nr-UE.

Figura 3.10. Registro del SMF sobre la ausencia de PCF y el uso de reglas locales.

Figura 3.11. Ping desde el nr-UE al UPF a través de la interfaz oaitun_ue1.

Figura 3.12. Ping desde el nr-UE a una dirección externa (verificación de N6).

Figura 3.13. Fragmento del values.yaml del AMF: bloque multus de la interfaz n2.

Figura 3.14. Fragmento del config.yaml del AMF: bloque nfs con la interfaz SBI y n2.

Figura 3.15. Fragmento del values.yaml del gNB: bloque multus de las interfaces n2 y n3.

Figura 3.16. Fragmento del values.yaml del gNB: bloque config y parámetro amfhost.

Figura 3.17. Fragmento del values.yaml del nr-UE: parámetro rfSimServer.

Figura 3.18. Tabla de gNB conectados publicada por el AMF.

Figura 3.19. Tabla de UE registrados publicada por el AMF.

Figura A.1. Interfaces de red de los Pods del 5GC y de la RAN levantados en el clúster.

Figura A.2. Puertos y conexiones relevantes para el enlace rfsimulator.

Figura A.3. Estado de los Pods del despliegue.

Figura A.4. Estado de los Deployments del despliegue.

Figura A.5. Estado de los PersistentVolumeClaims (PVC).

Figura A.6. Extracto de los registros del nr-UE en funcionamiento.

Figura A.7. Extracto de los registros del gNB en funcionamiento.

Figura A.8. Extracto de los registros del AMF en funcionamiento.

Figura A.9. Extracto de los registros del AUSF en funcionamiento.

Figura A.10. Extracto de los registros del NRF en funcionamiento.

Figura A.11. Extracto de los registros del SMF en funcionamiento.

Índice de tablas

Tabla 2.1. Principales interfaces del 5G SA y pila de protocolos asociada.

Tabla 5.1. Presupuesto detallado del proyecto.

Tabla 6.1. Evaluación del grado de cumplimiento de los objetivos específicos.

Glosario de acrónimos

Acrónimo	Significado
3GPP	3rd Generation Partnership Project — organismo de estandarización de las redes móviles
5GC	5G Core — núcleo de red 5G
5G SA	5G Standalone — 5G independiente, sin apoyo del núcleo 4G
5G NSA	5G Non-Standalone — 5G apoyado en el núcleo 4G (EPC)
5QI	5G QoS Identifier — identificador de calidad de servicio en 5G
A1	Interfaz O-RAN entre el Non-RT RIC y el Near-RT RIC
AMF	Access and Mobility Management Function
API	Application Programming Interface
AUSF	Authentication Server Function
CNF	Cloud-Native Network Function — función de red virtualizada empaquetada como contenedor
CNI	Container Network Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CU	Centralized Unit (gNB / O-RAN)
CUPS	Control and User Plane Separation
DN	Data Network
DU	Distributed Unit (gNB / O-RAN)
E2	Interfaz O-RAN entre el Near-RT RIC y los nodos de la RAN
ETSI	European Telecommunications Standards Institute
gNB	gNodeB — estación base de la red de acceso 5G NR
GTP-U	GPRS Tunnelling Protocol — User plane
HARQ	Hybrid Automatic Repeat reQuest
HTTP/2	Hypertext Transfer Protocol, versión 2
IP	Internet Protocol
K8s	Kubernetes
MAC	Medium Access Control (capa de la interfaz radio)

Acrónimo	Significado
MIB	Master Information Block
N1–N6	Puntos de referencia 3GPP entre UE, gNB, AMF, SMF y UPF
NAD	NetworkAttachmentDefinition (Multus/Kubernetes)
NAS	Non-Access Stratum
NF	Network Function — función de red
NFV	Network Functions Virtualisation
NGAP	NG Application Protocol
NG-RAN	Next Generation Radio Access Network
NR	New Radio — interfaz radio del 5G
NRF	Network Repository Function
OAI	OpenAirInterface
O-CU-CP / O-CU-UP	O-RAN Central Unit — Control Plane / User Plane
O-DU	O-RAN Distributed Unit
O1	Interfaz de gestión O-RAN entre el SMO y los nodos
OFH	Open Fronthaul — interfaz O-RAN entre O-DU y O-RU
O-RAN	Open Radio Access Network
O-RU	O-RAN Radio Unit
PBCH	Physical Broadcast Channel
PCI	Physical Cell Identity
PDCP	Packet Data Convergence Protocol
PDU	Protocol Data Unit
PFCP	Packet Forwarding Control Protocol
PHY	Physical layer — capa física
PSS / SSS	Primary / Secondary Synchronization Signal
QoS	Quality of Service — calidad de servicio
rApp	Aplicación del Non-RT RIC (O-RAN)
REST	Representational State Transfer
RIC	RAN Intelligent Controller
RLC	Radio Link Control

Acrónimo	Significado
RRC	Radio Resource Control
RT	Real Time — tiempo real
SBA	Service-Based Architecture — arquitectura basada en servicios
SBI	Service-Based Interface — interfaz basada en servicios
SCTP	Stream Control Transmission Protocol
SDR	Software Defined Radio — radio definida por software
SIB	System Information Block
SMF	Session Management Function
SMO	Service Management and Orchestration (O-RAN)
TCP	Transmission Control Protocol
TEID	Tunnel Endpoint Identifier
TFG	Trabajo de Fin de Grado
UDM	Unified Data Management
UDP	User Datagram Protocol
UDR	Unified Data Repository
UE	User Equipment — equipo de usuario
UPF	User Plane Function
USRP	Universal Software Radio Peripheral — familia de hardware SDR de Ettus Research
VM	Virtual Machine — máquina virtual

1. Introducción

1.1. Motivación

Durante mi periodo de prácticas académicas tuve la oportunidad de colaborar en una empresa dedicada al soporte de varias redes 5G. Esa experiencia me permitió observar de primera mano hasta qué punto las redes de telecomunicaciones están adoptando paradigmas cloud-native, y despertó mi interés por comprender en profundidad qué implica desplegar una red móvil sobre una plataforma de orquestación de contenedores. De esta experiencia surgió la idea de este Trabajo de Fin de Grado.

Actualmente en la industria de las telecomunicaciones se observa una transformación impulsada por el 5G, donde la separación explícita entre software y hardware es más evidente que nunca. La virtualización de las funciones de red y la adopción de contenedores orquestados mediante Kubernetes ha permitido la creación de arquitecturas basadas en microservicios que sustituyen el modelo tradicional basado en equipos dedicados y propietarios.

Según GSMA Intelligence, a finales de 2024 sesenta operadores habían desplegado redes 5G Standalone en producción y aproximadamente dos tercios de los operadores encuestados a escala mundial habían iniciado algún tipo de despliegue de 5G SA y a su vez este mismo crecimiento de las redes 5G está impulsando el crecimiento de las funciones de red cloud-native. Comprender esta transformación a un nivel más práctico mediante el despliegue de un sistema real resulta especialmente valioso para la formación de un ingeniero de telecomunicaciones. [36]

Existen, además, proyectos de código abierto (como OpenAirInterface, empleado en este trabajo) que facilitan el acceso a implementaciones completas y adaptadas a los estándares 3GPP de una red 5G, permitiendo desplegar bancos de pruebas funcionales sin depender de equipamiento comercial ni de licencias, gracias al uso de simuladores de radiofrecuencia como rfsimulator. Este trabajo lo he enmarcado en esa línea, aprovechando dichas herramientas para construir, documentar y depurar un despliegue completo de una red 5G Standalone sobre una infraestructura de contenedores gestionada mediante Kubernetes, replicando la arquitectura de red multi-interfaz que exige el estándar.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo general de este Trabajo de Fin de Grado es diseñar, desplegar y documentar una red 5G Standalone funcionalmente completa, tanto su núcleo de red como su acceso radio, sobre un clúster de Kubernetes de un único nodo, empleando componentes de código abierto de OpenAirInterface.

Se ha prestado especial atención a la interoperabilidad entre las distintas funciones de red (CNF) desplegadas y a la orquestación del conjunto mediante Kubernetes, validando su correcto

funcionamiento mediante el registro completo de un equipo de usuario software en el núcleo de red.

1.2.2. Objetivos específicos

El despliegue de una red 5G es un objetivo demasiado genérico, por lo que para poder hacer un mejor seguimiento del estado del trabajo de fin de grado he decidido dividir el objetivo general en múltiples objetivos específicos (OE), cada uno de estos objetivos se describe a continuación:

- OE1 — Desplegar el núcleo de red 5G (5GC) de OAI (AMF, SMF, UPF, NRF, AUSF, UDM y UDR) mediante Helm sobre un clúster Kubernetes gestionado con Minikube. Criterio de aceptación: los ocho Pods alcanzan el estado Running y cada NF se registra correctamente contra el NRF.
- OE2 — Configurar la segmentación de red multi-interfaz que exige el modelo de referencia 3GPP (interfaces N1 a N6) mediante Multus CNI y el plugin macvlan. Criterio de aceptación: cada NetworkAttachmentDefinition crea su interfaz sin conflictos de direccionamiento, verificable mediante ip a dentro de cada Pod.
- OE3 — Desplegar el gNB y el equipo de usuario software (nr-UE) de OAI, conectados mediante el módulo rfsimulator como sustituto de la radiofrecuencia real. Criterio de aceptación: conexión TCP de rfsimulator establecida entre ambos procesos, verificable en /proc/net/tcp.
- OE4 — Diagnosticar y resolver las incidencias técnicas que surjan durante la integración de los distintos componentes. Criterio de aceptación: cada incidencia queda documentada con su causa raíz, el método de diagnóstico empleado y la solución aplicada.
- OE5 — Validar el registro completo del equipo de usuario en el núcleo de red (sincronización de capa física, conexión RRC y registro NAS con el AMF) y la conectividad de datos de extremo a extremo. Criterio de aceptación: log de registro NAS exitoso en el AMF y respuesta positiva a un ping cursado a través de la interfaz oaitun_ue1.
- OE6 — Elaborar la presente memoria como documento técnico autocontenido. Criterio de aceptación: el documento incluye marco teórico, implementación, conclusiones y una bibliografía, y sirve de referencia reproducible sin necesidad de consultar fuentes externas adicionales.

Los cumplimientos asociados a cada uno de estos objetivos se evaluarán más adelante en el capítulo 6 de este documento.

1.3. Alcance

El presente trabajo se centra en la **validación funcional de una red 5G SA de un gNB y un UE únicos, sobre un clúster de Kubernetes de un único nodo**, y no persigue objetivos de rendimiento, escalabilidad ni alta disponibilidad, propios de un despliegue de producción.

Por ello es relevante remarcar que queda fuera del alcance:

- Medición de prestaciones radio o de red, throughput, latencia, capacidad, más allá de la comprobación básica de que la conectividad funciona.

- Uso de hardware de radiofrecuencia real, ya que toda la validación se apoya deliberadamente en el simulador software rfsimulator de OAI.
- Despliegue de una arquitectura O-RAN desagregada, con O-CU, O-DU, O-RU y RIC, que será descrita en el apartado 2.2.6 únicamente como marco teórico de referencia y como posible continuación natural de este trabajo.
- No se explota el network slicing u otras funcionalidades avanzadas del 5GC no esenciales para el registro y la conectividad básica del UE.

Por último, y en coherencia con el carácter de banco de pruebas académico de este despliegue, tampoco se persigue un despliegue multinodo de Kubernetes.

1.4. Estructura del documento

Una vez expuestos el contexto y el objetivo del documento, a continuación, se detalla la estructura que seguirá el contenido en los siguientes apartados:

Capítulo 2: Marco teórico
En este capítulo se desarrolla el marco teórico necesario para comprender las decisiones de diseño del despliegue: la arquitectura 5G SA y sus funciones de red, la pila de protocolos de la interfaz radio, las tecnologías de contenedores y orquestación (Docker, Kubernetes, Helm), las redes multi-interfaz en Kubernetes (Multus CNI, macvlan), el proyecto OpenAirInterface y la arquitectura O-RAN.

Capítulo 3: Diseño e implementación del despliegue
Aquí se describe el diseño, la implementación y el procedimiento completo de despliegue realizado: el entorno de trabajo, la arquitectura de red resultante, las incidencias técnicas encontradas y resueltas, la configuración del enlace radio simulado, el estado actual del despliegue, la metodología de depuración empleada y la guía paso a paso que permite reproducirlo.

Capítulo 4: Marco regulador
En esta sección se analiza el marco regulador aplicable (estándares técnicos, regulación del espectro radioeléctrico, seguridad de las redes 5G, licencias de software y protección de datos).

Capítulo 5: Entorno socioeconómico
Se presenta el presupuesto del proyecto, su impacto socioeconómico y su contribución a los Objetivos de Desarrollo Sostenible.

Capítulo 6: Conclusiones
En él se recogen las conclusiones del trabajo, evaluando el grado de cumplimiento de cada uno de los objetivos específicos planteados en este capítulo, y propone líneas futuras de continuación.

El documento se cierra con las referencias bibliográficas empleadas a lo largo de la memoria.

2. Marco teórico

2.1. Introducción a las redes móviles 5G

La quinta generación de redes móviles (5G) supone un cambio de paradigma respecto a las generaciones anteriores, no solo por el incremento de las prestaciones radio, sino por la propia arquitectura de red que la sustenta. Mientras que el LTE (4G) se diseñó en torno a una arquitectura de red relativamente monolítica, el 5G definido por el 3GPP se concibe desde el origen como una arquitectura de servicios (Service-Based Architecture, SBA), modular, virtualizable y desplegable sobre infraestructuras de computación en la nube.

El 3GPP contempla dos modos principales de despliegue de 5G: el modo No Independiente (Non-Standalone, NSA), en el que la nueva interfaz radio NR se apoya en un núcleo de red LTE (EPC) ya existente, y el modo Independiente (Standalone, SA), en el que tanto el acceso radio como el núcleo de red son íntegramente 5G.

Este Trabajo de Fin de Grado se centra en el modo 5G SA, por ser el que expresa de forma completa las capacidades arquitectónicas del 5G (arquitectura basada en servicios, network slicing, separación del plano de control y de usuario) y por ser el modo soportado de forma nativa por el proyecto OpenAirInterface.

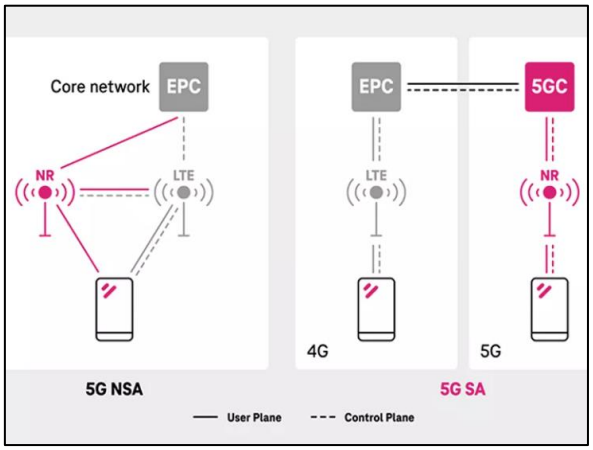


Figura 2.1. Comparativa de los modos de despliegue 5G No Independiente (NSA) e Independiente (SA). Fuente: T-Mobile (s.f.), "Las ventajas de seguridad de las redes 5G independientes"

A alto nivel, una red 5G SA se compone de tres bloques funcionales: el equipo de usuario (UE), la red de acceso radio de nueva generación (NG-RAN) y el núcleo de red 5G (5G Core, 5GC). Los siguientes apartados describen la arquitectura de cada uno de estos bloques con el nivel de detalle necesario para comprender las decisiones de diseño tomadas en el despliegue objeto de este trabajo.

2.1.1. Conceptos previos: cliente-servidor y transporte de datos

Para una mejor comprensión del resto de esta memoria es conveniente describir los conceptos de cliente-servidor y transporte de datos, pues serán especialmente relevantes más adelante.

El modelo cliente-servidor es aquel en el que una máquina permanece con un puerto a la espera de peticiones y un cliente inicia la comunicación. Este es el método que utilizan las NFs para comunicarse dentro del core 5G y el enlace rfsimulator que se describe en el apartado 2.2.2.3.

El segundo concepto que es relevante es diferenciar entre TCP, un protocolo de transporte basado en la fiabilidad, y UDP, otro protocolo que se basa en la velocidad a costa de que no garantiza la entrega de los paquetes. En general 5G opera mediante SCTP, una variante de TCP, para la señalización crítica y UDP para el plano de usuario y aquellas señalizaciones donde se gestionan los reintentos a nivel de aplicación. Esto será relevante más adelante para corrección de errores y depuración.

2.2. Arquitectura de la red 5G Standalone

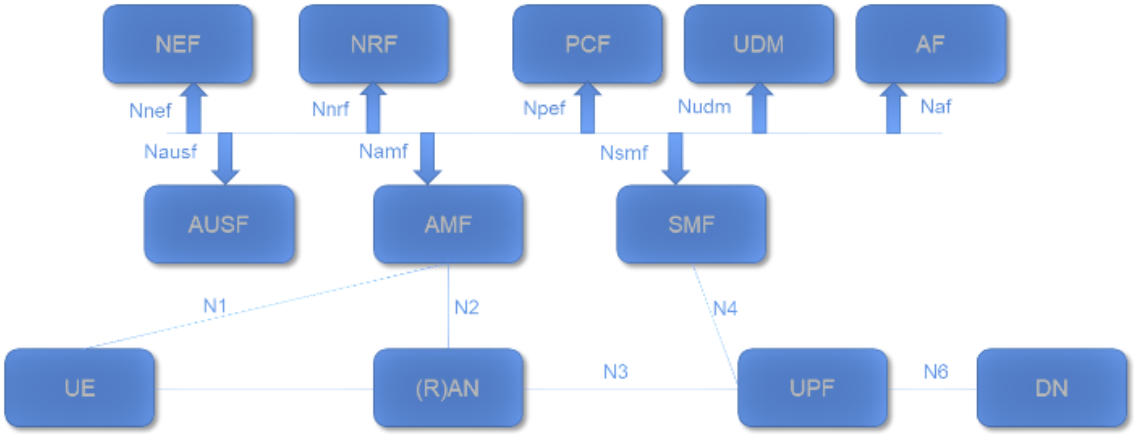


Figura 2.2. Arquitectura general de una red 5G Standalone: UE, NG-RAN y 5GC. (Fuente: Millán, R. (2019), "Tecnologías y funciones de la red troncal 5G", ramonmillan.com)

2.2.1. El núcleo de red 5G (5GC)

El núcleo de red 5G, especificado en 3GPP TS 23.501, cláusula 4.2 [1], abandona el modelo de nodos monolíticos de generaciones anteriores en favor de un conjunto de funciones de red (Network Functions, NF) independientes que se comunican entre sí mediante interfaces de servicio basadas en HTTP/2 y REST. Es decir, cada NF es un programa especializado en resolver una tarea muy concreta dentro de la red —por ejemplo, decidir si un usuario puede conectarse, o encaminar sus paquetes de datos— y varias de estas NF cooperan entre sí, cada una haciendo su parte, en lugar de un único programa monolítico que lo haga todo. Cada NF puede desplegarse, escalarse y actualizarse de forma independiente, lo que encaja de forma natural con tecnologías cloud-native como los contenedores y la orquestación mediante Kubernetes, descritos en los apartados 2.3 y 2.4.

A continuación, se describe individualmente cada una de las funciones de red que componen el 5GC, detallando su función y las interfaces que la conectan con el resto del sistema, siendo este

el conjunto de componentes que se despliega como Pods independientes en el clúster de Kubernetes objeto de este trabajo. La figura 2.2 resume esta arquitectura general de un clúster 5G, distinguiendo el plano de servicios (SBI) del plano de interfaces punto a punto.

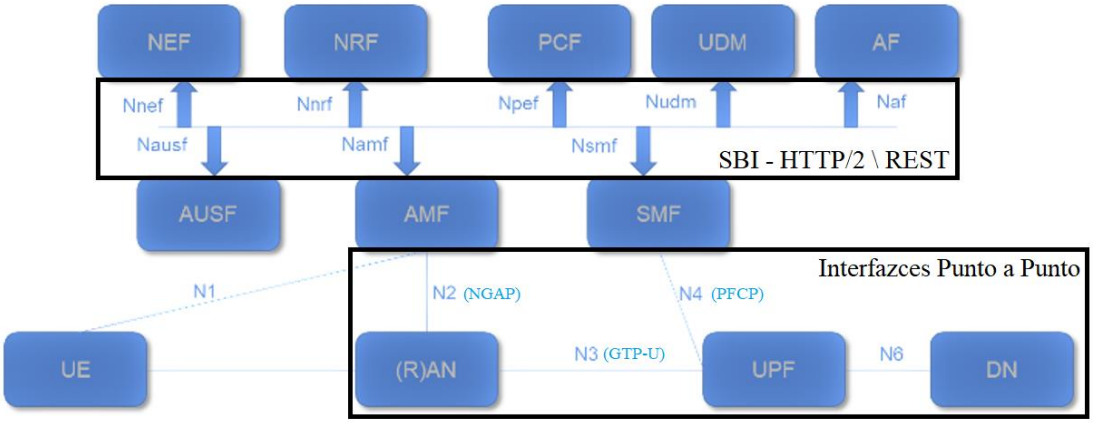


Figura 2.3. Funciones de red del 5GC: arquitectura basada en servicios (SBA). (Fuente: Millán, R. (2019), "Tecnologías y funciones de la red troncal 5G", ramonmillan.com)

2.2.1.1. AMF (Access and Mobility Management Function)

El AMF es la función de red que actúa como punto de entrada único de la señalización de control procedente tanto del UE (interfaz N1) como del gNB (interfaz N2).

Sus responsabilidades, definidas en 3GPP TS 23.501, cláusula 6.2.1 [1], incluyen:

1. La gestión del registro del UE en la red, definida en 3GPP TS 24.501, cláusula 5.5.1 [1], [13]

2. El control de la movilidad entre celdas, conforme a 3GPP TS 23.501, cláusula 5.3.4 [1]

3. El mantenimiento de la accesibilidad y del contexto de conexión, cuyos estados CM-IDLE y CM-CONNECTED se definen en 3GPP TS 23.501, cláusula 5.3.3 [1]

4. La coordinación de los procedimientos de seguridad (autenticación y establecimiento del contexto de seguridad NAS), delegando la verificación de credenciales en el AUSF

En el despliegue realizado en este TFG, el AMF es la network function con la que más interactúan tanto el gNB como el UE y por tanto una de las NFs que más información nos proporciona a lo largo de este proyecto ya que se encargará de confirmar la correcta conexión de estas dos funciones al resto del core 5G.

2.2.1.2. SMF (Session Management Function)

El SMF, cuyas funcionalidades se definen en 3GPP TS 23.501, cláusula 6.2.2 [1], gestiona el ciclo de vida completo de las sesiones PDU (Protocol Data Unit) del UE: su establecimiento, modificación y liberación.

Entre sus funciones se incluyen:

1. La asignación de direcciones IP al UE
2. La selección del UPF adecuado para cada sesión, en caso de existir varios
3. Su configuración mediante la interfaz N4
4. La aplicación de políticas de calidad de servicio (QoS): en 5G, cada flujo de tráfico de una sesión PDU se asocia a un identificador 5QI (5G QoS Identifier) que determina su prioridad, retardo máximo tolerable y tasa de pérdida de paquetes objetivo, de forma análoga a como las clases de servicio priorizan el tráfico en una red IP convencional. Estas políticas son recibidas de la PCF

El SMF no maneja tráfico de usuario en ningún momento pues delega el control de paquetes del plano de usuario en el UPF.

2.2.1.3. UPF (User Plane Function)

El UPF, definido en 3GPP TS 23.501, cláusula 6.2.3 [1], es la única función de red del 5GC que procesa el plano de usuario, es decir, el tráfico de datos real del UE. Actúa como punto de anclaje entre la red de acceso radio (interfaz N3, hacia el gNB) y las redes de datos externas (interfaz N6), y es responsable del encaminamiento y reenvío de paquetes, la aplicación de políticas de QoS por sesión mencionadas en la descripción del SMF, y la generación de informes de uso de tráfico. Su separación completa respecto al plano de control (principio conocido como CUPS, Control and User Plane Separation) permite escalar el UPF de forma independiente al resto de funciones, lo que resulta especialmente relevante en despliegues con requisitos de rendimiento o de baja latencia.

2.2.1.4. AUSF (Authentication Server Function)

El AUSF, definido en 3GPP TS 23.501, cláusula 6.2.8 [1], se encarga de la autenticación del 5GC, dando soporte a los métodos de autenticación 5G-AKA. Existen otros métodos mediante los cuales el AUSF puede autenticar como puede ser EAP-AKA' pero no son utilizados en este despliegue. Estos métodos de autenticación se recogen en más detalle en 3GPP *TS 33.501, cláusula 6.1.3* [14]. El AUSF recibe las solicitudes de autenticación procedentes del AMF y, apoyándose en los datos de suscripción y las claves criptográficas gestionadas por el UDM, verifica la identidad del UE y genera el material criptográfico necesario para el establecimiento del contexto de seguridad, devolviendo el resultado de la autenticación al AMF.

A continuación, muestro un esquema de los mensajes que se llevan a cabo durante una autenticación mediante 5G-AKA, el SEAF es una función lógica que está dentro del AMF:

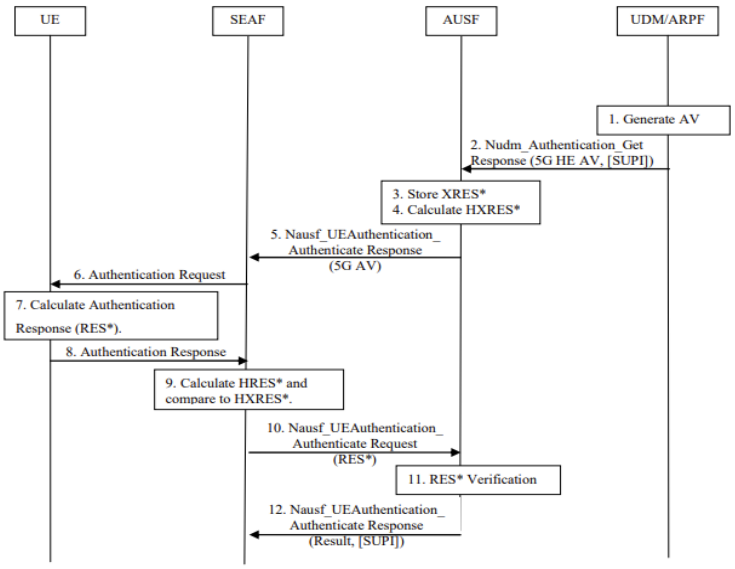


Figura 2.4. Procedimiento de autenticación 5G-AKA. Fuente: 3GPP TS 33.501, cláusula 6.1.3.2, figura 6.1.3.2-1 [14].

2.2.1.5. UDM (Unified Data Management)

El UDM, definido en 3GPP TS 23.501, cláusula 6.2.7 [1], gestiona todo aquello relativo a los suscriptores: perfiles de acceso y movilidad, generación de credenciales de autenticación (en colaboración con el AUSF) y gestión de la autorización de acceso a la red y a los servicios contratados. El UDM se encarga de negociar con el UDR todos estos datos pues no posee almacenaje persistente.

2.2.1.6. UDR (Unified Data Repository)

El UDR, definido en 3GPP TS 23.501, cláusula 6.2.11 [1], constituye el repositorio de datos centralizado del 5GC, y almacena tanto los datos de suscripción empleados por el UDM y el AUSF como los datos de política empleados por la PCF. El concentrar todo el almacenaje persistente en una única función permite que el resto de NFs del core sean, en su mayoría, funciones sin estado (stateless), lo que nos simplifica su escalado horizontal (levantar Pods del mismo tipo) en un entorno de contenedores.

2.2.1.7. NRF (Network Repository Function)

El NRF, definido en 3GPP TS 23.501, cláusula 6.2.6 [1], es el elemento central de la arquitectura basada en servicios (SBA). Este actúa como un DNS interno del 5GC en el que cada NF se registra al arrancar, indicando su tipo, dirección e interfaces de servicio disponibles, y frente al cual el resto de NFs pueden consultar las instancias del resto de NFs registradas (descubrimiento de servicios). Gracias a esto no es necesario configurar de forma estática las direcciones de cada NF en el resto de los componentes, y resulta ideal para entornos donde los Pods pueden reiniciarse o cambiar de dirección IP interna, como ocurre en Kubernetes.

2.2.2. La red de acceso radio de nueva generación (NG-RAN)

La red de acceso, especificada en 3GPP TS 38.401, cláusula 6 [4], está formada por nodos gNB que implementan la interfaz radio NR (New Radio) y se conectan al núcleo de red mediante la interfaz N2 (plano de control, hacia el AMF) y N3 (plano de usuario, hacia el UPF).

El 3GPP contempla la posibilidad de dividir funcionalmente el gNB en unidades separadas, CU (Centralized Unit), DU (Distributed Unit) y, opcionalmente, RU (Radio Unit), comunicadas mediante las interfaces F1, E1 y fronthaul, lo que habilita despliegues desagregados propios de arquitecturas O-RAN [3], aunque en este trabajo se ha optado por emplear un gNB monolítico.

A continuación, se describen individualmente los dos componentes de la RAN presentes en el despliegue, el gNB y el UE software, así como el módulo rfsimulator, que sustituye al canal radio real y constituye el punto de unión entre ambos.

2.2.2.1. gNB (gNodeB)

El gNB es el nodo de acceso radio de la red 5G NR. Su papel y la pila de protocolos que implementa se describen en 3GPP TS 38.300, cláusula 4 [3]: en su versión monolítica incorpora la pila completa de la interfaz radio, esto es, PHY (capa física), MAC, RLC, PDCP y RRC.

Sus funciones incluyen:

1. La generación y transmisión de las señales de sincronización (PSS/SSS) y del canal de difusión (PBCH, con el MIB) que permiten a los UEs detectar y sincronizarse con la celda.

2. La programación (scheduling) de los recursos radio en los dominios de tiempo y frecuencia

3. La gestión de las conexiones RRC de los UEs asociados

4. La interconexión con el núcleo de red a través de las interfaces N2 (señalización con el AMF) y N3 (plano de usuario con el UPF)

La configuración del gNB incluye parámetros de celda cuya definición y rangos admisibles se recogen en 3GPP TS 38.104 [9], como la banda de frecuencia, la numerología (el conjunto de valores de espaciado entre subportadoras y duración de símbolo que NR permite seleccionar, a diferencia de LTE, que emplea un único espaciado fijo de 15 kHz), el ancho de banda y el identificador de celda física (physCellId), del que depende la correcta decodificación del PBCH por parte del UE.

2.2.2.2. nr-UE (equipo de usuario software)

El nr-UE es la implementación software, proporcionada por OAI, del equipo de usuario (UE) definido por el 3GPP.

Al igual que el gNB, implementa la pila de protocolos radio completa desde el punto de vista del terminal: realiza la búsqueda y sincronización de celda mediante la detección de las señales PSS/SSS, cuyo procedimiento se define en 3GPP TS 38.213, cláusula 4.1 [7]; decodifica el MIB a través del PBCH [5], [8]; establece la conexión RRC con el gNB conforme a 3GPP TS 38.331, cláusula 5.3.3 [8]; y ejecuta la señalización NAS de registro y gestión de sesión contra el núcleo de red a través del AMF y del SMF [13]. Una vez completado el registro y el establecimiento de la sesión PDU, el nr-UE asigna la dirección IPv4 recibida en el elemento de información PDU

address del mensaje PDU Session Establishment Accept, definido en 3GPP TS 24.501, cláusula 9.11.4.10 [13], a la interfaz túnel oaitun_ue1 creada durante la inicialización, sobre la que se cursa el tráfico de datos de usuario de forma análoga a como lo haría la interfaz de datos móviles de un terminal real.

2.2.2.3. rfsimulator: simulación del canal radio

El módulo rfsimulator, descrito en el propio github de OAI [30], sustituye la capa de radiofrecuencia física por una conexión software basada en sockets TCP/IP entre los procesos de gNB y UE. Por encima de esta capa de simulación, toda la pila de protocolos radio se ejecuta de forma idéntica a como lo haría con radiofrecuencia real, lo que permite validar el comportamiento funcional completo del sistema sin necesidad de equipamiento SDR ni de un entorno de laboratorio con aislamiento radioeléctrico.

La pila de protocolos es igual entre el gNB y UE y esta definida por el 3GPP para la interfaz radio NR, se organiza en las siguientes capas:

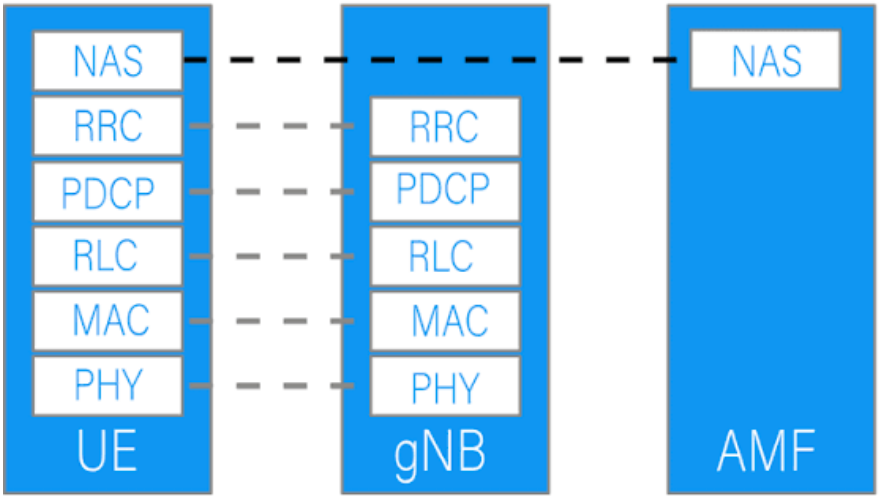


Figura 2.5. Pila de protocolos de la interfaz NR y alcance del rfsimulator. [48]

PHY: se encarga de la modulación, la codificación de canal y las señales de sincronización;

MAC: se encarga del acceso al medio y de la programación de recursos radio;

RLC: se encarga de segmentar y reensamblar los paquetes;

PDCP: se encarga del cifrado, la protección de integridad y la entrega ordenada;

RRC: se encarga del establecimiento y la reconfiguración de la conexión radio.

NAS: conecta al UE con el AMF.

El rfsimulator sustituye la capa PHY, las demás capas se ejecutan exactamente igual que con radiofrecuencia real.

Para el correcto funcionamiento de rfsimulator se necesita configurar los dos procesos (gNB y UE) para que uno de ellos actue como servidor y el otro como cliente. Lo mas habitual es

configurar el gNB como servidor TCP mediante el parámetro serveraddr =”server” y el UE como cliente mediante el parámetro rfsimulator.serveraddr que debería apuntar a la ip del gNB.

Una vez configurado rfsimulator las muestras que habitualmente viajarían por el canal radio se transmiten por esta conexión TCP, por lo que el correcto funcionamiento del simulador depende de que ambos procesos puedan producir y consumir las muestras dentro de unos márgenes de tiempo que impone la interfaz NR.

El hecho de que rfsimulator dependa del procesamiento en tiempo real estricto es, precisamente, la razón por la que la documentación oficial de OAI desaconseja de forma explícita la ejecución de gNB y UE sobre máquinas virtuales pues la latencia y sobre todo las interrupciones (jitter) introducidas por la capa de virtualización y por el planificador de CPU del hipervisor pueden provocar que los procesos no completen a tiempo la generación o el consumo de muestras.

Esto se manifiesta habitualmente como saturación del buffer de envío (Send-Q) del socket de rfsimulator. Esta circunstancia, detectada durante el desarrollo de este TFG y descrita con más detalle en el capítulo 3, constituye la hipótesis de partida para explicar los fallos de sincronización observados y motiva las medidas de configuración de tiempo real (kernel RT y la deshabilitación de estados C/P) adoptadas en el entorno de despliegue.

2.2.3. Interfaces y flujo de registro del UE

El procedimiento de acceso de un UE a la red es conceptualmente similar a sintonizar una emisora de radio y una vez sintonizada, escuchar el boletín inicial que resume la programación del canal: primero hay que encontrar la frecuencia y el instante exactos en los que engancharse a la señal (sincronización) y solo después se puede interpretar el contenido que se transmite sobre ella. De forma más precisa:

1. En primer lugar, el UE realiza la sincronización de capa física con la celda a través de las señales PSS/SSS (Primary/Secondary Synchronization Signal), definidas en 3GPP TS 38.211, cláusula 7.4.2 [5], que le permiten obtener el identificador físico de celda (Physical Cell ID, PCI) y la referencia temporal

2. A continuación, decodifica el canal de difusión PBCH (Physical Broadcast Channel), definido en 3GPP TS 38.211, cláusula 7.4.3 [5], y cuya codificación se especifica en 3GPP TS 38.212, cláusula 7.1 [6]. El PBCH transporta el bloque maestro de información del sistema (MIB), cuyo contenido se define en 3GPP TS 38.331, cláusula 6.2.2 [8].

3. Para que la decodificación del PBCH sea correcta, el receptor debe descodificar la señal utilizando exactamente el mismo identificador de celda (Nid_cell) que emplea el transmisor.

4. Tras completar la sincronización de capa física y la decodificación de MIB/PBCH el UE puede proceder a establecer la conexión RRC, conforme a 3GPP TS 38.331, cláusula 5.3.3 [8], y posteriormente el registro NAS, especificado en 3GPP TS 24.501, cláusula 5.5.1 [13], y cuyo flujo completo entre funciones de red se recoge en 3GPP

TS 23.502, cláusula 4.2.2.2.2 [2], con el AMF a través de la interfaz N1/N2, paso final que habilita la creación de la interfaz de datos del UE (en OAI, la interfaz oaitun_ue) para el tráfico de usuario.

2.2.4. Interfaces y protocolos de comunicación

El 3GPP define de forma precisa, mediante puntos de referencia identificados con la letra N, cada uno de los enlaces lógicos que conectan las funciones de red entre sí, enumerados en 3GPP TS 23.501, cláusula 4.2.3 [1], especificando para cada uno el protocolo de aplicación y la pila de transporte empleados: NGAP sobre SCTP para N2 [10], GTP-U sobre UDP para N3 [11] y PFCP sobre UDP para N4 [12]. La tabla 2.1 resume las interfaces relevantes para el despliegue realizado en este trabajo.

Interfaz	Extremos	Protocolo de aplicación	Transporte
N1	UE ↔ AMF	NAS	Encapsulado en RRC / NGAP (sin transporte IP propio)
N2	gNB ↔ AMF	NGAP	SCTP / IP
N3	gNB ↔ UPF	GTP-U	UDP / IP
N4	SMF ↔ UPF	PFCP	UDP / IP
N6	UPF ↔ DN externa	Tráfico de usuario (IP nativo)	IP
Nnrf / Namf / Nsmf / Nudm / Nudr / Nausf	Resto de NFs ↔ NF correspondiente (SBA)	API REST	HTTP/2 / TCP / IP

Tabla 2.1. Principales interfaces del 5G SA y pila de protocolos asociada.

En esta tabla se pueden diferenciar claramente dos tipos de interfaces:

1. Las que se heredan del modelo clásico punto a punto siendo estas las interfaces N2, N3 y N4; cada una con su propio protocolo de señalización, a estas también se le añade la interfaz N6, la salida del UPF hacia la red externa.

2. Las interfaces del plano de servicios (SBI) que se identifican por el prefijo N seguida del nombre de la NF (Nnrf, Namf, …), estas sustituyen la señalización punto apunto por llamadas a servicios REST sobre HTTP/2, operando de manera similar a una API web moderna.

Esta distinción tiene una consecuencia directa sobre la arquitectura de red del despliegue descrita en el capítulo 3: las interfaces N2, N3, N4 y N6, por transportar señalización y plano de usuario sensibles a la segmentación de red propia de un despliegue 3GPP realista, se implementan cada una como una interfaz de red macvlan independiente gestionada por Multus. Las interfaces SBI, en cambio, al tratarse de llamadas HTTP/2 entre microservicios, se cursan con

normalidad a través de la red por defecto del clúster (eth0) y de los Service/ClusterIP asociados a cada NF, sin requerir una interfaz de red dedicada adicional.

2.2.5. SCTP como transporte de la señalización N2

Los protocolos de aplicación descritos en la tabla 2.1 (NGAP, PFCP, las API de servicio) no viajan directamente sobre IP, sino sobre un protocolo de transporte intermedio, de forma análoga a como HTTP viaja sobre TCP en la web. De todos ellos, el que más condiciona el comportamiento observado durante la depuración descrita en el capítulo 3 es SCTP.

SCTP es el protocolo de transporte elegido por el 3GPP [10] para la interfaz N2. Retomando la distinción entre TCP y SCTP introducida en el apartado 2.1.1, SCTP puede entenderse como un punto intermedio: es fiable como TCP, pero está orientado a mensajes en lugar de a flujo continuo de bytes, y añade multi-streaming y multi-homing, dos características pensadas para redes de señalización.

En OAI, la asociación SCTP N2 se establece entre la dirección macvlan del gNB y la dirección macvlan del AMF. La comprobación del estado de la conexión es verificable mediante el fichero /proc/net/sctp/eps del núcleo Linux, es el primer hito de conectividad comprobado en el capítulo 3 (apartado 3.3.2).

2.2.6. O-RAN: hacia una RAN abierta y desagregada

O-RAN Alliance [33] es una asociación de operadores, fabricantes y centros de investigación cuyo objetivo es transformar la red de acceso radio en un ecosistema abierto, desagregado e inteligente, complementando las especificaciones 3GPP descritas en los apartados anteriores.

Mientras que el 3GPP define la posibilidad de dividir el gNB en CU y DU y estandariza la interfaz F1 entre ambas, O-RAN Alliance extiende esa desagregación hasta la propia unidad de radio (O-RU) y añade un conjunto de interfaces abiertas y de controladores inteligentes que no forman parte del 3GPP, con el objetivo de romper la dependencia de soluciones propietarias verticales y permitir el despliegue multi-vendor de los distintos componentes.

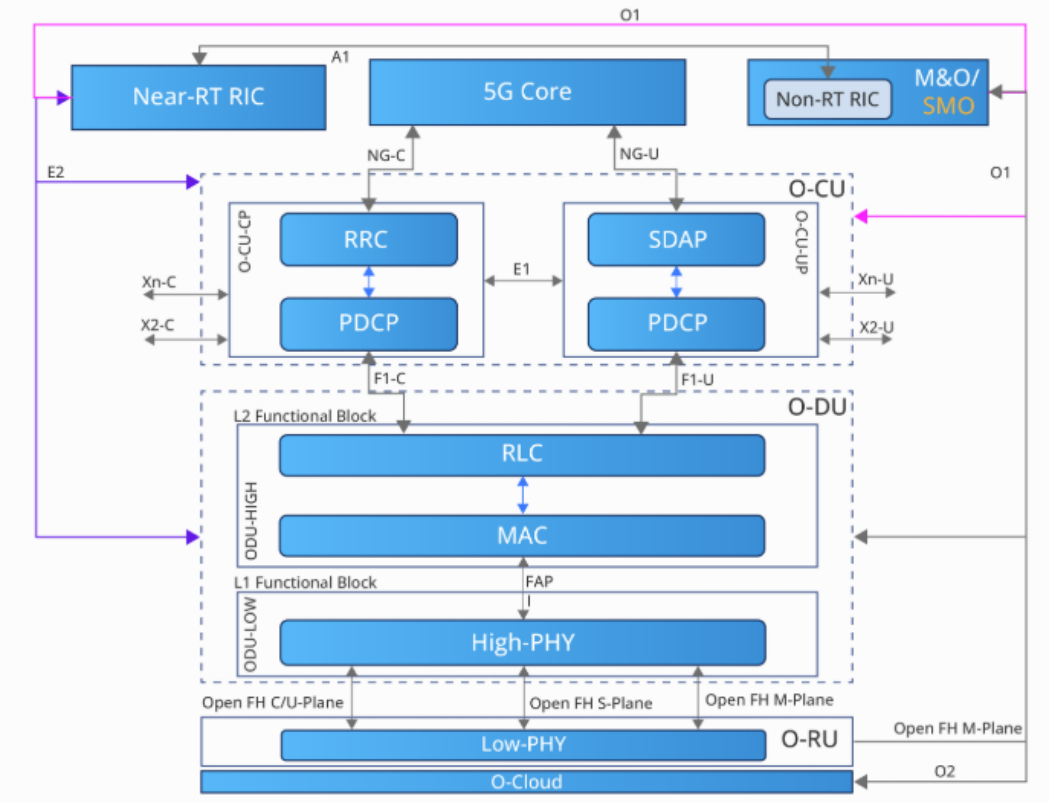


Figura 2.6. Arquitectura O-RAN: desagregación del gNB y RAN Intelligent Controllers. [34]

La arquitectura O-RAN [34], resumida en la figura 2.6, mantiene la desagregación 3GPP del gNB en O-CU-CP, O-CU-UP y O-DU y añade el O-RU como elemento independiente, conectado al O-DU mediante la interfaz Open Fronthaul (OFH), definida íntegramente por O-RAN Alliance para garantizar la interoperabilidad entre unidades de radio y unidades distribuidas de distintos fabricantes.

El despliegue realizado en este TFG no implementa la arquitectura O-RAN: el gNB de OAI se emplea en su forma monolítica, sin división en O-CU/O-DU/O-RU ni RIC alguno, por ser la configuración soportada de forma nativa y más directa para la validación funcional que constituye el objetivo de este trabajo. No obstante, el propio proyecto OpenAirInterface proporciona también componentes RAN desagregados conformes a O-RAN (CU-CP, CU-UP y DU independientes), documentados en trabajos que integran dichos componentes con núcleos de red de terceros [31], lo que sitúa a OAI como una base habitual tanto para despliegues monolíticos como para bancos de pruebas O-RAN completos, y constituye una vía natural de trabajo futuro para extender el despliegue de este TFG.

2.3. Virtualización de funciones de red y arquitecturas cloud-native

2.3.1. De las máquinas virtuales a los contenedores

Antes de describir la adopción cloud-native del 5GC es recomendable explicar qué es un contenedor y en qué se diferencia de una máquina virtual (VM), dado que ambos son mecanismos de virtualización, pero funcionan de manera diferente.

Una máquina virtual simula el hardware completo a través de un hipervisor (en el caso de este trabajo, Proxmox): emula una máquina física completa sobre la que se instala un sistema operativo invitado independiente y completo, con su propio kernel. Esto aporta un aislamiento muy fuerte entre VMs, pero representa una sobrecarga considerable dado que cada VM ejecuta su propio kernel, lo que consume memoria y CPU adicionales y ralentiza el arranque.

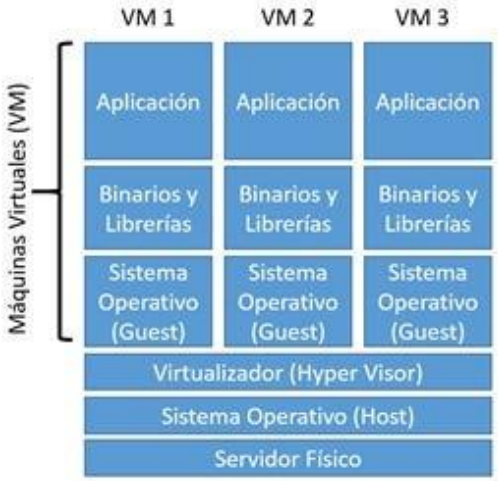


Figura 2.7. Arquitectura de una máquina virtual: cada VM ejecuta su propio sistema operativo invitado sobre el hipervisor.

Un contenedor, en cambio, virtualiza a nivel del sistema operativo, es decir, todos los contenedores de una misma máquina comparten el mismo kernel del sistema anfitrión y el aislamiento entre ellos se consigue mediante el propio kernel del sistema anfitrión, en lugar de mediante la emulación de hardware. El resultado es una ejecución mucho más ligera que una VM, con tiempos de arranque de milisegundos o pocos segundos frente a los minutos de una VM, lo que permite una densidad de despliegue muy superior en la misma infraestructura física.



Figura 2.8. Arquitectura de un contenedor: los contenedores comparten el kernel del sistema operativo anfitrión.

Los contenedores en este trabajo se crean y ejecutan utilizando Docker. Cuando Minikube emplea el driver de Docker (apartado 2.4.3), el propio nodo de Kubernetes se ejecuta como un contenedor Docker, y cada Pod de Kubernetes se implementa, a su vez, como uno o varios contenedores Docker adicionales gestionados por el kubelet.

2.3.2. Adopción cloud-native en el núcleo de red 5G

La adopción de arquitecturas cloud-native en el ámbito de las telecomunicaciones se enmarca dentro de la evolución impulsada por ETSI NFV (Network Functions Virtualisation) [15], que propuso sustituir los equipos de red dedicados (appliances hardware) por funciones de red.

Esta tendencia ha evolucionado desde las máquinas virtuales hacia los contenedores descritos en el apartado anterior, considerados actualmente el estándar de facto para el despliegue de funciones de red 5G debido a su menor sobrecarga, mayor densidad de despliegue y tiempos de arranque y escalado reducidos.

Cuando una función de red virtualizada (VNF) se empaqueta específicamente como contenedor en lugar de como máquina virtual, se emplea el término más específico CNF (Cloud-Native Network Function): cada NF del 5GC descrita en el apartado 2.2.1 es una CNF, y es precisamente esta naturaleza la que permite que Kubernetes las trate como cualquier otra carga de trabajo cloud-native, sin necesidad de mecanismos de orquestación específicos para telecomunicaciones.

El 5GC, por su propia arquitectura basada en servicios (SBA), es especialmente adecuado para esta virtualización ligera: cada NF puede empaquetarse como una imagen de contenedor independiente, con su propio ciclo de vida, y comunicarse con el resto de funciones mediante API REST/HTTP2 (apartado 2.2.4), en lugar de mediante protocolos de señalización propietarios entre nodos hardware.

2.4. Kubernetes como plataforma de orquestación de contenedores

Las CNFs si bien tienen muchas ventajas el ejecutarlas sin un orquestador de contenedores como Kubernetes resulta ineficiente pues mantener sin un orquestador la cantidad de Pods necesarios para una red 5G de manera continuada sería prácticamente imposible.

En este trabajo, Kubernetes, se encarga de arrancar los Pods en el orden correcto, reiniciarse si fallan, encontrarse entre sí y recibir la configuración adecuada, tarea que como ya he mencionado no resultia práctica de gestionar a mano.

2.4.1. Arquitectura general de Kubernetes

Kubernetes es la plataforma de orquestación de contenedores de facto en el sector, y automatiza el despliegue, escalado y operación de aplicaciones contenerizadas [16]. Su función puede entenderse como la de un supervisor automático: el usuario declara cuántas copias de cada programa quiere tener funcionando y con qué recursos, y Kubernetes se encarga de decidir en qué máquina física arrancarlas, de reiniciarlas si fallan, y de mantener ese estado deseado de

forma continua sin intervención manual, tarea que sería inviable de gestionar a mano en un sistema con decenas de componentes como el 5GC. Su arquitectura distingue entre el plano de control (control plane), formado por el API server (punto de entrada único a todas las operaciones sobre el clúster), etcd (base de datos clave-valor donde se persiste el estado del clúster), el scheduler (encargado de decidir en qué nodo se ejecuta cada Pod) y el controller manager (que reconcilia continuamente el estado real del clúster con el estado deseado declarado por el usuario); y los nodos de trabajo (worker nodes), en cada uno de los cuales se ejecuta el kubelet (agente que gestiona el ciclo de vida de los contenedores en ese nodo) y un runtime de contenedores (en este trabajo, Docker). A continuación se describen los recursos y herramientas concretos de este ecosistema empleados en el despliegue realizado.

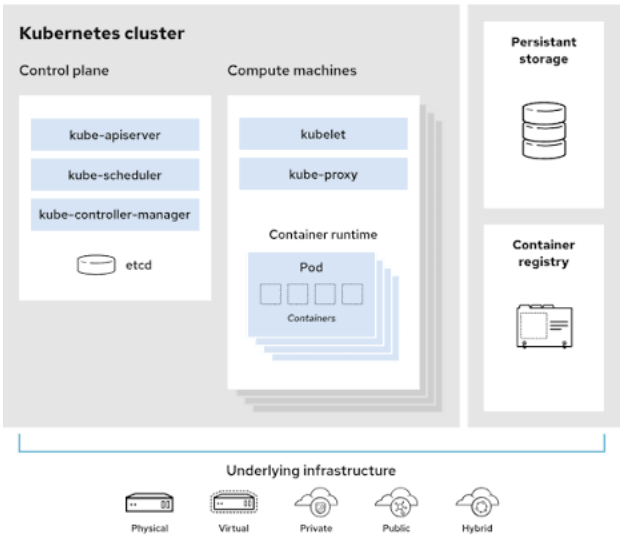


Figura 2.9. Arquitectura general de Kubernetes (clúster de un único nodo).

2.4.1.1. Pod

El Pod [21] es la unidad mínima de despliegue de Kubernetes. Agrupa uno o más contenedores que comparten espacio de nombres de red (network namespace, en el sentido del mecanismo de aislamiento del kernel descrito en el apartado 2.3) y, por tanto, dirección IP, puertos y opcionalmente de almacenamiento.

En el despliegue de este TFG, cada función de red del 5GC (AMF, SMF, UPF, etc.) y cada componente de la RAN (gNB, nr-UE) se ejecuta como un Pod independiente con uno o dos contenedores, dependiendo de si se levanta el contenedor para tcpdump, lo que permite tratar cada elemento de la arquitectura 3GPP como una unidad de despliegue, reinicio y depuración aislada.

2.4.1.2. Deployment

Un Deployment [22] es un recurso de nivel superior que gestiona el ciclo de vida de un conjunto de Pods idénticos a través de un ReplicaSet subyacente: garantiza que el número de réplicas deseado esté siempre en ejecución, sustituyendo automáticamente los Pods que fallen, y gestiona las actualizaciones progresivas (rolling updates) cuando cambia la definición del Pod (por ejemplo, al actualizar la imagen de un contenedor). Cada una de las funciones de red desplegadas en este TFG se gestiona mediante un Deployment con una única réplica, dado que,

al tratarse de un banco de pruebas académico de validación funcional, no se requiere alta disponibilidad ni escalado horizontal.

2.4.1.3. Service

El Service [17] es la abstracción de Kubernetes que proporciona una dirección IP estable (ClusterIP) y balanceo de carga hacia un conjunto de Pods, desacoplando a los clientes de la dirección IP concreta de cada Pod individual.

En este despliegue, los Services se emplean para las comunicaciones internas de Kubernetes y para las llamadas de servicio HTTP/2 de la arquitectura basada en servicios del 5GC (por ejemplo, el registro de cada NF contra el NRF), pero no para la señalización 3GPP sobre las interfaces N1/N2/N3/N4, que se cursa directamente por las interfaces macvlan gestionadas por Multus, al margen del Service/ClusterIP de Kubernetes, tal y como se detalla en el apartado 3.3.3.

2.4.1.4. PersistentVolume y PersistentVolumeClaim (PV/PVC)

Dado que el sistema de ficheros de un contenedor es efímero y se destruye al eliminarse el Pod, Kubernetes desacopla el almacenamiento persistente mediante dos recursos [23]: el PersistentVolume (PV), que representa una porción de almacenamiento físico o de red disponible en el clúster, y el PersistentVolumeClaim (PVC), mediante el cual un Pod solicita y reserva parte de ese almacenamiento sin necesidad de conocer los detalles de la infraestructura subyacente.

En el despliegue de OAI, este mecanismo se emplea principalmente para persistir la base de datos de suscriptores del UDR, de modo que los datos de aprovisionamiento del UE sobrevivan a un reinicio del Pod correspondiente.

2.4.1.5. ConfigMap y Secret

Los ConfigMap [24] y los Secret [25] permiten desacoplar la configuración de una aplicación de la imagen del contenedor que la ejecuta, inyectándola en tiempo de despliegue como variables de entorno o ficheros montados dentro del Pod.

Los charts de Helm de OAI hacen un uso extensivo de ConfigMap para generar, a partir de los valores definidos en values.yaml, los ficheros de configuración de cada función de red y componente de la RAN (direcciones IP, parámetros de celda, credenciales, etc.), lo que constituye el mecanismo concreto por el cual las modificaciones de configuración descritas en el capítulo 3 (por ejemplo, la dirección de escucha del AMF) se trasladan al proceso en ejecución.

2.4.1.6. Namespace

El Namespace [26] permite segmentar lógicamente los recursos de un mismo clúster físico en particiones independientes desde el punto de vista de nombres y de políticas de acceso.

En este trabajo se trabaja principalmente sobre el namespace default, pero es necesario conocer la existencia de este recurso principalmente para comprobar el estado general del clúster de Kubernetes y del plugin Multus dado que los Pods que se encargan de estas dos funciones se encuentran en un namespace aparte (kube-system).

2.4.2. kubectl: interfaz de línea de comandos

kubectl [27] es la herramienta de línea de comandos oficial para interactuar con el API server de Kubernetes, y constituye la vía principal empleada en este trabajo tanto para el despliegue (kubectl apply, o de forma indirecta a través de Helm) como para la operación y depuración del sistema mediante:

1. La inspección del estado de los Pods (kubectl get pods, kubectl describe pod),
2. El acceso a sus registros (kubectl logs)
3. La ejecución de comandos dentro de un contenedor en ejecución (kubectl exec), esta última especialmente utilizada durante la depuración descrita en el capítulo 3 para inspeccionar desde dentro de cada Pod el estado de las interfaces de red y de los sockets (por ejemplo, mediante ifconfig, ss o la lectura directa de /proc/net/sctp/eps).

2.4.3. Minikube: distribución de Kubernetes de un único nodo

Minikube [20] es una distribución de Kubernetes pensada para la ejecución de un clúster completo (plano de control y nodo de trabajo) sobre una única máquina, orientada a entornos de desarrollo, pruebas y aprendizaje en los que no se dispone de una infraestructura multinodo.

Minikube admite distintos drivers para la creación del nodo (máquina virtual, contenedor Docker, etc.); en este trabajo se emplea el driver de Docker, de modo que el propio nodo de Kubernetes se ejecuta como un contenedor Docker dentro de la máquina virtual Debian que aloja el despliegue.

Esta configuración de un único nodo simplifica la gestión del clúster, pero implica que todos los Pods comparten los mismos recursos de CPU y el mismo planificador del sistema operativo anfitrión.

2.4.4. Helm como gestor de paquetes

Helm [18] es el gestor de paquetes de Kubernetes, y organiza las definiciones de recursos (Deployments, Services, ConfigMaps, NetworkAttachmentDefinitions, etc.) en unidades reutilizables denominadas charts.

Un chart de Helm emplea plantillas parametrizables mediante un fichero values.yaml, siendo yaml un formato de texto estructurado que permite instanciar la misma definición de despliegue con configuraciones distintas (direcciones IP, interfaces de red, parámetros radio, réplicas, etc.) sin duplicar manifiestos.

En el despliegue de OAI sobre Kubernetes, cada función de red del 5GC y cada componente de la RAN (gNB, UE) se distribuye como un chart de Helm independiente, lo que facilita tanto la instalación como la reconfiguración incremental del sistema durante las fases de depuración.

2.5. Redes avanzadas en Kubernetes: Multus CNI

El modelo de red estándar de Kubernetes, definido por la especificación CNI (Container Network Interface) [16], asigna a cada Pod una única interfaz de red (además de loopback) conectada a

la red plana del clúster. Este modelo, adecuado para aplicaciones web genéricas, resulta insuficiente para funciones de red de telecomunicaciones, que necesitan distinguir de forma explícita entre planos de red diferenciados: señalización de control (por ejemplo, N2 entre gNB y AMF), plano de usuario (N3, entre gNB y UPF) y red de gestión del propio clúster.

Multus CNI [19] resuelve esta limitación actuando como un meta-plugin capaz de invocar varios plugins CNI para un mismo Pod. De forma similar a como un ordenador físico puede tener varias tarjetas de red, un Pod gestionado por Multus puede tener varias interfaces de red independientes, cada una conectada a un segmento de red distinto. Con Multus, cada Pod conserva su interfaz de red por defecto (eth0, gestionada por el CNI principal del clúster) y puede añadir interfaces de red secundarias adicionales, cada una definida mediante un recurso propio de Kubernetes denominado NetworkAttachmentDefinition (NAD).

En el despliegue realizado en este TFG, cada interfaz N2 (AMF-gNB) y N3 (UPF-gNB) del 5GC se implementa como una interfaz de red secundaria independiente mediante Multus, mientras que el plugin macvlan se emplea como plugin de bajo nivel para cada NAD. Macvlan crea, sobre una interfaz física o virtual del nodo, subinterfaces virtuales con su propia dirección MAC, visibles como interfaces de red de primer nivel dentro del Pod, lo que permite asignarles direcciones IP estáticas dentro de un segmento de red propio para cada plano (N1/N2, N3, N4, N6), replicando así la segmentación de interfaces característica de un despliegue real de un núcleo 5G.

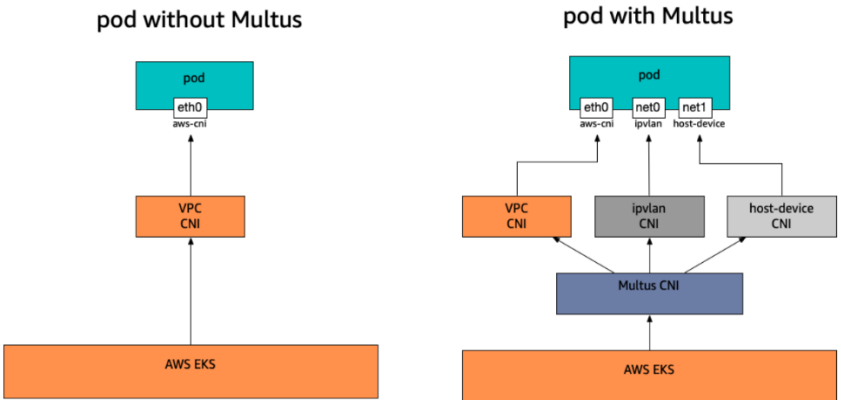


Figura 2.10. Ejemplo de cómo funciona Multus CNI y plugin macvlan. El ejemplo es en AWS pero sigue el mismo principio. [49]

Esta segmentación, si bien es fiel a la arquitectura de referencia del 3GPP, introduce una complejidad operativa considerable en un clúster de un único nodo como el usado en este trabajo: procesos que por defecto escuchan en la interfaz eth0 del Pod deben configurarse explícitamente para enlazar sus sockets a la interfaz macvlan correspondiente, y debe garantizarse la unicidad de las direcciones IP estáticas asignadas a cada NAD para evitar conflictos que impidan la creación de la propia interfaz. Ambas cuestiones, identificadas durante el desarrollo de este TFG, se detallan en el capítulo de implementación.

2.6. El proyecto OpenAirInterface (OAI)

Todo lo descrito hasta ahora (la arquitectura 5G SA, los contenedores, Kubernetes, Multus) son piezas de infraestructura genérica, aplicables en principio a cualquier sistema distribuido; falta todavía lo que las convierte específicamente en una red 5G: el software que implementa, de forma conforme a los estándares, cada una de las funciones descritas en el apartado 2.2. Ese papel lo cumple, en este trabajo, el proyecto OpenAirInterface.

OpenAirInterface (OAI) es un proyecto de software abierto, mantenido por la OpenAirInterface Software Alliance (OSA), que proporciona una implementación software de la pila de protocolos 3GPP [28] tanto para la red de acceso radio como para el núcleo de red, cubriendo generaciones LTE y 5G. Su naturaleza abierta y su conformidad con las especificaciones 3GPP y O-RAN lo han convertido en una de las referencias más utilizadas por la comunidad académica e investigadora para el despliegue de bancos de pruebas 5G de bajo coste, sin depender de equipamiento comercial cerrado.

2.6.1. OAI 5G Core (oai-cn5g)

El proyecto OAI CN5G [28] proporciona una implementación conforme a 3GPP de un núcleo de red 5G SA con arquitectura basada en servicios, incluyendo las funciones AMF, SMF, UPF, NRF, AUSF, UDM y UDR. Cada función de red se distribuye como una imagen de contenedor independiente y dispone de su propio chart de Helm, lo que permite su despliegue tanto mediante Docker Compose —orientado a entornos de un único host— como mediante Kubernetes, empleando Helm charts oficiales del proyecto, opción utilizada en este TFG.

2.6.2. OAI RAN: gNB y nr-UE

El proyecto OAI RAN [29] implementa el gNB (oai-gnb) y el equipo de usuario software (oai-nr-ue) descritos en el apartado 2.2.2, distribuidos igualmente como imágenes de contenedor con sus correspondientes charts de Helm (oai-gnb, oai-nr-ue), lo que permite integrarlos en el mismo clúster de Kubernetes que el núcleo de red y gestionarlos con las mismas herramientas de despliegue.

2.7. Trabajos relacionados

El despliegue de redes 5G sobre Kubernetes empleando componentes de OpenAirInterface ha sido documentado en diversos entornos experimentales y bancos de prueba académicos [31].

Estos despliegues se se han hecho tanto en configuraciones desagregadas donde hacían uso de núcleos de terceros como puede ser Open5GS, como en despliegues monolíticos similares al de este trabajo.

La literatura y documentación disponible coincide en señalar la gestión de las interfaces de red secundarias mediante Multus/macvlan y el correcto enlazado de sockets a dichas interfaces como uno de los puntos de mayor dificultad práctica en este tipo de despliegues, lo que concuerda con las incidencias documentadas y resueltas durante el desarrollo de este TFG.

También es destacable que los bancos de pruebas basados en OAI, tanto sobre infraestructuras de investigación como sobre entornos de un único nodo [32], tienden a utilizar rfsimulator como alternativa de bajo coste a la radiofrecuencia real, lo que apoya la decisión de utilizarlo para trabajos de carácter académico como el presente.

3. Diseño e implementación del despliegue

3.1. Entorno de trabajo

El despliegue de este trabajo, como ya se ha indicado con anterioridad, se realiza sobre un único nodo de Kubernetes; este nodo ha sido levantado mediante Minikube, utilizando Docker como el controlador subyacente. Además de usar Minikube también es necesaria la instalación del plugin Multus CNI para poder establecer las diferentes interfaces dentro de los Pods que desplegaremos en este trabajo. Por último, las network functions tanto del 5GC como las relacionadas con el gNB y el UE de la RAN serán desplegadas mediante el uso de la herramienta HELM. Si bien la mayor parte de las veces se pueden desplegar las funciones de manera conjunta haciendo uso de unos archivos llamados Charts en los directorios descargados de OAI, a veces, como puede ser el caso del gNB o del UE, se deberán desplegar de manera individual llamando al archivo values.yaml que existe dentro de los directorios individuales de cada función. Esto último también es relevante si quisiéramos levantar una NF específica sin afectar al resto.

La elección de levantar toda la red 5G en un único nodo, si bien simplifica respecto a un clúster multinodo como los que se suelen encontrar en el mundo laboral, es suficiente para validar el comportamiento de 5G SA y reduce la complejidad operativa durante el desarrollo y depuración. No obstante, también introduce particularidades propias de Minikube que han causado ciertos cambios en el despliegue con respecto a los ejemplos dados por OAI.

3.1.1. De un despliegue en máquina física con USRP a un entorno virtualizado con rfsimulator

El despliegue utilizado como base para este trabajo estaba configurado originalmente para funcionar haciendo uso de un USRP. Esto, para la situación de este trabajo de fin de grado, resulta muy inconveniente, pues me he visto forzado a usar una máquina virtual cedida por la universidad, dado que no tengo máquinas físicas con las especificaciones suficientes como para ejecutar el clúster 5G, lo que complicaría la conexión del USRP pues se debería conectar a nivel de nodo y carezco de los permisos para estas acciones y además, el uso de un USRP implica un gasto económico considerable pues son material de investigación bastante caro.

Teniendo todo esto en cuenta, decidí que el mejor curso de acciones para este trabajo de fin de grado sería hacer uso de la herramienta rfsim que proporciona Open Air Interface para simular las conexiones radio tanto del UE como del gNB. Esta decisión, si bien resuelve algunos problemas, también me ha causado otros a lo largo de este trabajo pues aunque ya no tengo que configurar el USRP aparte con todo lo que ello implicaría sigo teniendo el problema de no tener un control completo sobre el nodo donde está montada la máquina virtual lo que me ha generado problemas principalmente con las velocidades del CPU y el jitter generado al compartir el procesamiento de una misma CPU entre diferentes máquinas virtuales ajenas a mi proyecto.

Por último, mencionar que el hecho de haber utilizado rfsim en vez de un USRP me ha forzado a tener que modificar de manera considerable tanto el ConfigMap del UE como el del gNB pues como ya he mencionado antes estos archivos estaban pensados para su ejecución haciendo uso de un USRP, más detalles sobre estas modificaciones en el apartado 3.4.

3.2. Arquitectura de red del despliegue

Siguiendo el modelo de interfaces del 3GPP, cada plano de comunicación del sistema se implementa mediante una interfaz de red Multus independiente, definida como un NetworkAttachmentDefinition (NAD) con plugin macvlan y dirección IP estática:

- N2 (AMF ↔ gNB): señalización de control NGAP sobre SCTP, mediante los NAD oai-amf-n2 y oai-gnb-n2.
- N3 (UPF ↔ gNB): plano de usuario GTP-U.
- N4 (SMF ↔ UPF): señalización PFCP.
- N6: salida del UPF hacia la red de datos externa.

Cada Pod conserva además su interfaz eth0 por defecto, gestionada por el CNI principal del clúster, utilizada únicamente para las comunicaciones internas de Kubernetes (API server, resolución DNS interna, etc.), pero no para la señalización 3GPP entre funciones de red, que debe cursarse explícitamente por las interfaces macvlan. A continuación, muestro tanto las interfaces del AMF como el NAD de la interfaz N2 del AMF como ejemplo de cómo están levantadas las diferentes interfaces.

```
===== pod/oai-amf-56946b77bc-c8dbl =====
Defaulted container "tcpdump" out of: tcpdump, amf, init (init)
eth0      Link encap:Ethernet  HWaddr 66:78:B6:A2:DD:01
          inet addr:10.244.0.187  Bcast:10.244.255.255  Mask:255.255.0.0
          inet6 addr: fe80::6478:b6ff:fea2:dd01/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:286388 errors:0 dropped:0 overruns:0 frame:0
          TX packets:364839 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:27818705 (26.5 MiB)  TX bytes:32003552 (30.5 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

n2        Link encap:Ethernet  HWaddr DE:4B:0F:E5:C5:4E
          inet addr:172.21.6.95  Bcast:172.21.7.255  Mask:255.255.252.0
          inet6 addr: fe80::dc4b:fff:fee5:c54e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:23922 errors:0 dropped:0 overruns:0 frame:0
          TX packets:23996 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2108110 (2.0 MiB)  TX bytes:2112188 (2.0 MiB)
```

Figura 3.1. Interfaces de red del Pod oai-amf: eth0 (CNI principal), lo y n2 (macvlan). Salida del comando kubectl exec -it oai-amf-XXXXXXX -- ifconfig.

```
Name:         oai-amf-n2
Namespace:    default
Labels:       app.kubernetes.io/managed-by=Helm
Annotations:  meta.helm.sh/release-name: basic
              meta.helm.sh/release-namespace: default
API Version:  k8s.cni.cncf.io/v1
Kind:         NetworkAttachmentDefinition
Metadata:
  Creation Timestamp:  2026-08-15T23:27:07Z
  Generation:          1
  Resource Version:    548
  UID:                 7debd61b-f8c3-4f03-9f1a-ee3b01031eac
Spec:
  Config:  { "cniVersion": "0.3.1", "type": "macvlan", "master":"eth0", "mode": "bridge", "ipam": { "type": "static", "addresses": [ { "address":"172.21.6.95/22" } ] ,"routes":[{"dst":"10.8.0.0/24","gw":"172.21.7.254"}] } }
Events:    <none>
```

Figura 3.2. Definición del NetworkAttachmentDefinition oai-amf-n2. Salida del comando kubectl describe NAD oai-amf-n2.

En esta imagen se puede ver claramente cómo existen diferentes interfaces dentro del Pod oai-amf, siendo estas eth0 (la interfaz Ethernet original del Pod), lo (la interfaz de loopback) y n2, la interfaz generada mediante Multus, que está descrita más en detalle en la descripción del NAD, donde podemos ver cómo claramente es del tipo macvlan, que su master es eth0 y

que tiene una IP estática (172.21.6.95). La configuración del resto de Pods se encuentra en el Anexo 1 para aquellos interesados.

3.3. Resolución de incidencias en la fase de despliegue del núcleo

Durante el despliegue de la red 5G SA de OAI surgieron múltiples incidencias que considero resultan destacables. En este apartado se recopilan dichas incidencias, cómo se determinó la causa de estas y cómo fueron solucionadas.

3.3.1. Conflicto de direcciones IP estáticas duplicadas

Durante el despliegue inicial se detectó que la interfaz macvlan N2 no llegaba a crearse en el Pod del gNB. Al analizar la incidencia se detectó que los NAD oai-amf-n2 y oai-gnb-n2 tenían asignada, por error de configuración descargada de OAI, la misma dirección IP estática. Este error fue complicado de detectar inicialmente pues, si bien la solución es sencilla, Multus no genera en este caso un error explícito y visible de inmediato, sino que la interfaz simplemente no llega a instanciarse correctamente, lo que convierte este tipo de conflicto en un fallo silencioso. La solución consistió en verificar la unicidad de las direcciones IP asignadas a cada NAD del despliegue antes de la instalación y la consecuente reasignación de direcciones IP mediante el values.yaml o, en caso de no estar preparado, también se podría modificar el ConfigMap directamente, pues el values.yaml es un archivo que se utiliza para modificar otros archivos como el ConfigMap, que es el relevante en este caso. A continuación, muestro el fragmento de código del values.yaml y en negrita la IP que se modificó.

```
oai-amf:
  enabled: true
  kubernetesType: Vanilla #Openshift/Vanilla Vanilla for Upstream Kubernetes
  nfimage:  # image name either locally present or in a public/private repository
    repository: docker.io/oaisoftwarealliance/oai-amf           ## The image will be pulled from dockerhub
    version: v2.0.1                                    ## The branch to be used to pull from dockerhub
    # pullPolicy: IfNotPresent or Never or Always
    pullPolicy: IfNotPresent
  includeTcpDumpContainer: true #only for taking pcaps inside network function pod
  imagePullSecrets:
    - name: "regcred"
  multus:
    ## If you don't want to add a default route in your pod then replace this field with ""
    defaultGateway: ""
    n2Interface:
      create: true
      Ipadd: "172.21.6.95"
      Netmask: "22"
      # #name inside the pod is hardcoded right now
      # name: "n2"
      ## If you do not have a gateway leave the field empty
      #Gateway:
      ## If you do not want to add any routes in your pod then leave this field empty
      routes: [{'dst': '10.8.0.0/24','gw': '172.21.7.254'}]
      hostInterface: "eth0" # Interface of the host machine on which this pod will be scheduled
  nodeSelector: {}
```

Figura 3.3. Fragmento del values.yaml del chart oai-amf con la dirección IP estática asignada al NAD de la interfaz N2.

3.3.2. Enlazado del socket SCTP del AMF a la interfaz macvlan

Resuelto el conflicto de direcciones y con la interfaz macvlan ya visible en ambos Pods, cabría esperar que la asociación SCTP entre gNB y AMF se estableciera sin más contratiempos; sin embargo, seguía sin producirse. La inspección del socket mediante /proc/net/sctp/eps mostró que el proceso del AMF continuaba enlazado a la dirección de la interfaz eth0 del Pod en lugar de a la dirección de su interfaz macvlan N2, pese a que esta última se había creado correctamente. Esto se debe a que, por defecto, los binarios de OAI enlazan sus sockets a la primera interfaz de red disponible del contenedor si no se indica explícitamente lo contrario. La solución aplicada fue forzar de forma explícita, mediante los parámetros de configuración de config.yaml, el parámetro interface_name como dirección de escucha del AMF para la interfaz N2, dado que originalmente estaba configurada como eth0. La complicación de esta incidencia residió en que la modificación necesaria se encontraba en el config.yaml de la función en específico, siendo una de las pocas incidencias que no se pudo solucionar desde la carpeta principal del despliegue del core (./oai-5g-core/oai-5g-basic). A continuación, muestro el fragmento de código que se modificó dentro del config.yaml.

```
nfs:
  amf:
    host: oai-amf
    sbi:
      port: 80
      api_version: v1
      interface_name: eth0
    n2:
      interface_name: n2
      port: 38412
```

Figura 3.4. Fragmento del ConfigMap del AMF: declaración de la interfaz n2 y del puerto 38412 para la asociación SCTP.

3.3.3. Registro del gNB frente al AMF

Con el socket SCTP correctamente enlazado, todo apuntaba a que el registro del gNB en el AMF sería inmediato; en su lugar, apareció un tercer obstáculo, esta vez del lado del gNB: estaba configurado para apuntar a la dirección ClusterIP del Service de Kubernetes del AMF, en lugar de a la dirección macvlan de su interfaz N2. Al tratarse de un plano de señalización aislado mediante Multus, el tráfico SCTP N2 no debe cursarse a través del Service/ClusterIP de Kubernetes, sino directamente contra la dirección IP macvlan del AMF. A continuación, muestro la clara diferencia que había entre la dirección IP del Service y la de la interfaz N2 generada mediante Multus, y el fragmento del values.yaml del gNB que cambié.

```
danielcarballo@TFEDCR:~/TFG/2oIntento/OAI_5G_SA_Helmchart_kubernetes/oai-5g-ran/oai-gnb$ kubectl get svc oai-amf
NAME      TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)             AGE
oai-amf   ClusterIP   10.102.27.33   <none>        80/TCP,38412/SCTP   17d
danielcarballo@TFEDCR:~/TFG/2oIntento/OAI_5G_SA_Helmchart_kubernetes/oai-5g-ran/oai-gnb$ kubectl exec -it oai-amf-56946b77bc-c8dbl -c amf -- ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.244.0.187  netmask 255.255.0.0  broadcast 10.244.255.255
        inet6 fe80::6478:b6ff:fea2:dd01  prefixlen 64  scopeid 0x20<link>
        ether 66:78:b6:a2:dd:01  txqueuelen 1000  (Ethernet)
        RX packets 377705  bytes 36675787 (36.6 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 480680  bytes 42165323 (42.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

n2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.21.6.95  netmask 255.255.252.0  broadcast 172.21.7.255
        inet6 fe80::dc4b:fff:fee5:c54e  prefixlen 64  scopeid 0x20<link>
        ether de:4b:0f:e5:c5:4e  txqueuelen 1000  (Ethernet)
        RX packets 31469  bytes 2774428 (2.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 31567  bytes 2779970 (2.7 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figura 3.5. Servicio ClusterIP del AMF e interfaces del Pod, mostrando la coexistencia de la dirección de servicio y la dirección macvlan de N2.

```
config:
  timeZone: "Europe/Paris"
  useAdditionalOptions: "--sa --continuous-tx --rfsim"
  gnbName: "FGE-USTHB"
  mcc: "001"    # check the information with AMF, SMF, UPF
  mnc: "01"     # check the information with AMF, SMF, UPF
  tac: "1"      # check the information with AMF
  sst: "1"  #currently only 4 standard values are allowed 1,2,3,4
  usrp: rfsim #allowed values rfsim, b2xx, n3xx or x3xx
  amfhost: "172.21.6.95"  # amf ip-address or service-name oai-amf-svc or 172.21.6.94
  n2IfName: "n2"     # if multus.n2Interface.create is true then use n2
  n3IfName: "n3"    #if multus.n3Interface.create is true then use n3 or you can only use 1 interface n2 or eth0
```

Figura 3.6. Fragmento del values.yaml del chart oai-gnb con el parámetro amfhost apuntando a la dirección macvlan del AMF.

Tras corregir el parámetro amf_ip_address en el values.yaml del gNB para apuntar a la dirección IP de n2, el registro NGAP se completó correctamente. Es destacable mencionar que uno de los fallos con las direcciones IP estáticas estuvo en el AMF, motivo por el cual el comentario de los desarrolladores de OAI alude a una ip diferente a la que se acabó poniendo.

3.4. Configuración del enlace radio simulado (rfsimulator)

En ausencia de hardware SDR (apartado 2.2.2.3), la comunicación entre el gNB y el UE se establece mediante el módulo rfsimulator de OAI, que requiere que uno de los dos extremos actúe como servidor TCP. En la configuración inicial ambos procesos quedaban por defecto en modo cliente, lo que impedía silenciosamente el establecimiento de la conexión. La solución consistió en fijar explícitamente el ConfigMap del gNB en modo servidor (serveraddr = "server") y en configurar el UE. Con esta configuración se confirmó, mediante inspección de /proc/net/tcp dentro del Pod del gNB, el establecimiento correcto de la conexión TCP entre ambos procesos, como se muestra a continuación:

```
danielcarballo@TFEDCR:~/TFG/2oIntento/OAI_5G_SA_Helmchart_kubernetes/oai-5g-ran/oai-nr-ue$ kubectl exec -it oai-gnb-86dd67f548-jm6zf -- cat /proc/net/tcp
  sl  local_address rem_address   st tx_queue rx_queue tr tm->when retrnsmt   uid  timeout inode
   0: 00000000:0FCB 00000000:0000 0A 00000000:00000000 00:00000000 00000000     0        0 101440176 1 000000001a547ce2 100 0 0 10 0
   1: 5E0615AC:0FCB 1B0615AC:E220 01 00000000:000681C0 00:00000000 00000000     0        0 222357181 1 000000003bc414f6 20 4 0 553 553
```

Figura 3.7. Estado de las conexiones TCP en el Pod del gNB. Salida del comando cat /proc/net/tcp, donde el valor st = 1 confirma el establecimiento del enlace rfsimulator.

Una vez rfsimulator está establecido, el UE completa correctamente la sincronización de capa física de la siguiente manera:

1. Detecta de forma estable los picos de correlación de las señales PSS/SSS,
2. Decodifica el bloque maestro de información (MIB) transportado en el PBCH
3. Progresa hasta el establecimiento de la conexión RRC.

Es conveniente indicar que el equipo de usuario es capaz de localizar el bloque SSB de forma autónoma, sin necesidad de especificar su posición mediante parámetros adicionales. Recalco este punto pues en su momento fue una de las hipótesis sobre la no conexión del UE al AMF.

Una vez se sincronizaron correctamente, se manifestó la incidencia más laboriosa de todo el despliegue. Los registros del gNB mostraban de forma recurrente el mensaje write() failed, errno(11), correspondiente al código EAGAIN sobre el socket TCP del rfsimulator: el buffer de envío del núcleo se saturaba y el sistema operativo rechazaba la escritura, aunque originalmente desconocía el motivo del error.

El diagnóstico se apoyó en una aparente contradicción. Las estadísticas de radio publicadas periódicamente por el gNB no presentaban errores, mientras el plano de usuario permanecía inoperativo y el canal de señalización SRB1 quedaba congelado en un contador de bytes constante.

Esta diferencia confirma que la cadena de procesado de banda base era correcta: las muestras que llegaban se decodificaban sin ningún error; de estos datos se supuso que el error consistía en que llegaban menos muestras de las que el receptor necesitaba. El fallo, por tanto, no estaba en la lógica de capa física ni en la parametrización radio, sino en el transporte que sustituye al enlace aéreo.

Para intentar solucionar este problema se hicieron varias cosas desde la reducción del paralelismo de hilos del gNB mediante el parámetro parallel_config, la activación del muestreo a tres cuartos de tasa con la opción -E hasta la reducción del ancho de banda de la portadora, pero el único cambio que consiguió estabilizar el enlace fue la ampliación de los buffers de socket del núcleo, aplicada tanto sobre la VM Debian como en el interior del contenedor de Minikube mediante los siguientes comandos:

```
sudo sh -c 'echo 67108864 > /proc/sys/net/core/wmem_max'

sudo sh -c 'echo 67108864 > /proc/sys/net/core/rmem_max'

sudo sh -c 'echo 16777216 > /proc/sys/net/core/wmem_default'

sudo sh -c 'echo 16777216 > /proc/sys/net/core/rmem_default'
```

Los dos primeros comandos elevan el tamaño máximo de los buffers de envío y recepción desde los 212 kB configurados por defecto en Debian hasta 64 MB, mientras que los dos siguientes fijan el valor inicial en 16 MB.

Con este margen, el núcleo absorbe en memoria las ráfagas producidas cuando el hipervisor retrasa la ejecución del proceso consumidor, en lugar de rechazar la escritura. El jitter de planificación sigue existiendo, pero deja de traducirse en pérdida de muestras.

El cuello de botella ocurría a causa del jitter y no era culpa de cómo se había implementado la red 5G, sino de los problemas asociados a utilizar una máquina virtual como la estructura donde montamos el clúster. Este hecho causaba que al no tener permisos para modificar la configuración del nodo no pudiéramos hacer CPU pinning o passthrough de CPU, lo que nos dejaba como única opción ajustar los buffers desde el sistema invitado. Esta modificación consiguió resolver el problema.

Es importante mencionar que estos valores en los buffers no persisten tras un reinicio del sistema.

3.5. Registro del UE y establecimiento de la sesión PDU

Estabilizado el enlace radio, el UE completa la totalidad del procedimiento de acceso definido por el 3GPP. A continuación, el UE inicia el procedimiento 5GSM de establecimiento de sesión PDU descrito en 3GPP TS 23.502.

La traza del SMF documenta la secuencia completa desde la recepción del Nsmf_PDUSession_CreateSMContext procedente del AMF donde ocurre la asignación de la dirección 12.1.1.100 del pool configurado para el DNN oai, pasando por el establecimiento de la sesión PFCP sobre la interfaz N4 y la generación del contenedor N1 con el mensaje PDU Session Establishment Accept y del contenedor N2 con el PDU Session Resource Setup Request dirigido al gNB, hasta la transición final del estado a PDU_SESSION_ACTIVE.

```
[2026-09-05 02:12:38.902] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.902] [smf_app] [info] Add PDU Session with Id 10
[2026-09-05 02:12:38.902] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.902] [smf_app] [info] Found UPF for this PDU session: 192.168.24.2
[2026-09-05 02:12:38.902] [smf_app] [info] Get default QoS for a PDU Session, key 1
[2026-09-05 02:12:38.902] [smf_app] [info] Get default QoS rule for a PDU Session (PDU session type 1)
[2026-09-05 02:12:38.903] [smf_app] [info] Handle N4 Session Establishment Response (PDU Session Id 10)
[2026-09-05 02:12:38.903] [smf_app] [info] Get default QoS Rule this PDU Session (ID 10)
[2026-09-05 02:12:38.903] [smf_n1 ] [info] Create N1 SM Container, PDU Session Establishment Accept
[2026-09-05 02:12:38.903] [smf_n1 ] [info] PDU_SESSION_ESTABLISHMENT_ACCEPT, encode starting...
[2026-09-05 02:12:38.903] [smf_app] [info] Get default QoS Flow Description (PDU session type 1)
[2026-09-05 02:12:38.903] [smf_n1 ] [info] Encode PDU Session Establishment Accept
[2026-09-05 02:12:38.903] [smf_n2 ] [info] Create N2 SM Information, PDU Session Resource Setup Request Transfer
[2026-09-05 02:12:38.904] [smf_app] [info] PDU_SESSION_ESTABLISHMENT_UE_REQUESTED
[2026-09-05 02:12:38.904] [smf_app] [info] Update PDU Session Status
[2026-09-05 02:12:38.904] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.904] [smf_app] [info] Set PDU Session Status to PDU_SESSION_ESTABLISHMENT_PENDING
[2026-09-05 02:12:38.904] [smf_app] [info] Set PDU Session Status to PDU_SESSION_ESTABLISHMENT_PENDING
[2026-09-05 02:12:38.904] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.904] [smf_app] [info] Set PDU Session UpCnxState to UPCNX_STATE_DEACTIVATED
[2026-09-05 02:12:38.913] [smf_api] [info] Handle PDU Session Update SM Context Request.
[2026-09-05 02:12:38.913] [smf_api] [info] Received a PDUSession_UpdateSMContext Request from AMF.
[2026-09-05 02:12:38.913] [smf_app] [info] Handle a PDU Session Update SM Context Request from an AMF (HTTP vers
ion 2)
[2026-09-05 02:12:38.913] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.913] [smf_app] [info] Handle a PDU Session Update SM Context Request message from an AMF (H
TTP version 2)
[2026-09-05 02:12:38.913] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.913] [smf_app] [info] PDU Session Resource Setup Response Transfer
[2026-09-05 02:12:38.913] [smf_n2 ] [info] Decode NGAP message (PDUSessionResourceSetupResponseTransfer) from N2
 SM Information
[2026-09-05 02:12:38.913] [smf_app] [info] PDU Session Establishment Request, processing N2 SM Information
[2026-09-05 02:12:38.913] [smf_app] [info] Find PDU Session with ID 10
[2026-09-05 02:12:38.914] [smf_app] [info] Handle N4 Session Modification Response (PDU Session Id 10)
[2026-09-05 02:12:38.914] [smf_app] [info] PDU Session Establishment Request (UE-Initiated)
[2026-09-05 02:12:38.914] [smf_app] [info] Set PDU Session Status to PDU_SESSION_ACTIVE
```

Figura 3.8. Traza del SMF durante el establecimiento de la sesión PDU: asignación de la dirección 12.1.1.100 y transición final a PDU_SESSION_ACTIVE.

El nr-UE, al recibir el Accept, extrae la dirección del elemento de información PDU address y configura mediante llamadas la interfaz túnel oaitun_ue1, que queda operativa con direccionamiento 12.1.1.100/24.

```
===== pod/oai-nr-ue-668664b8c-g7fpc =====
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.244.0.190  netmask 255.255.0.0  broadcast 10.244.255.255
        inet6 fe80::bc30:ebff:fe12:8067  prefixlen 64  scopeid 0x20<link>
        ether be:30:eb:12:80:67  txqueuelen 1000  (Ethernet)
        RX packets 1011  bytes 71681 (71.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 213  bytes 13196 (13.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

net1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.21.6.27  netmask 255.255.252.0  broadcast 172.21.7.255
        inet6 fe80::e488:d7ff:feec:2090  prefixlen 64  scopeid 0x20<link>
        ether e6:88:d7:ec:20:90  txqueuelen 1000  (Ethernet)
        RX packets 7954771482  bytes 20955690153288 8 (209.5 TB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 7546389199  bytes 20952992478288 6 (209.5 TB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

oaitun_ue1: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST>  mtu 1500
        inet 12.1.1.100  netmask 255.255.255.0  destination 12.1.1.100
        inet6 fe80::875d:5d28:4113:c057  prefixlen 64  scopeid 0x20<link>
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00  txqueuelen 500  (UNSPEC)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 11  bytes 528 (528.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figura 3.9. Interfaces de red del Pod del nr-UE, con la interfaz túnel oaitun_ue1 operativa y direccionamiento 12.1.1.100/24.

El error PCF SM Policy Association Creation was not successful, generado porque el SMF intentaba descubrir una función PCF ausente del despliegue. La solución no consiste en desplegar dicha función, innecesaria para el alcance de este trabajo, sino en indicar al SMF que emplee reglas PCC locales mediante el parámetro use_local_pcc_rules: yes de su configuración.

```
[2026-09-06 21:49:27.134] [smf_n7 ] [info] PCF SM Policy Association Creation was not successful. Continue using
 default rules
```

Figura 3.10. Registro del SMF indicando que la asociación de política con la PCF no se completa y que se continúa con reglas locales.

Es importante también la mención del hecho de que el gNB y el nr-UE mantienen una conexión entre sí que no se reconstruye de forma automática. Esto representa una limitación importante siempre que se reinicie el Pod del gNB, pues provoca que el nr-UE intente seguir conectándose al Pod del gNB de antes del reinicio causando así un fallo en el que pese a que toda la configuración tanto del gNB como del nr-UE son correctas la conexión fallará. Dado que Kubernetes no garantiza un orden de arranque entre deployments el procedimiento correcto para garantizar el funcionamiento es levantar siempre el AMF seguido del gNB y por último el nr-UE. En caso de que el AMF o el gNB se reinicien, es recomendable reiniciar el resto de los Pods mencionados en este orden. En un despliegue de producción existen métodos para automatizar este proceso, pero no están implementados en OAI.

Para comprobar el correcto funcionamiento de la red 5G una vez desplegada, OAI recomienda pruebas mediante tráfico ICMP, introduciéndonos en el Pod nr-UE, utilizamos el comando ping para intentar acceder mediante la interfaz oaitun_ue1 a la dirección del UPF. Este tráfico ICMP atraviesa toda la pila desplegada y por tanto la respuesta correcta a este tráfico constituye la verificación de que el plano de usuario y el plano de control se encuentran operativos, confirmando el objetivo principal del trabajo. Es recomendable también probar una dirección IP externa como podría ser el DNS de Google para comprobar la conexión N6.

```
danielcarballo@TFEDCR:~/TFG/evidencias$ cat ping_upf.log
PING 12.1.1.1 (12.1.1.1) from 12.1.1.100 oaitun_ue1: 56(84) bytes of data.
64 bytes from 12.1.1.1: icmp_seq=1 ttl=64 time=6.58 ms
64 bytes from 12.1.1.1: icmp_seq=2 ttl=64 time=5.06 ms
64 bytes from 12.1.1.1: icmp_seq=3 ttl=64 time=4.57 ms
64 bytes from 12.1.1.1: icmp_seq=4 ttl=64 time=6.01 ms
64 bytes from 12.1.1.1: icmp_seq=5 ttl=64 time=5.45 ms
64 bytes from 12.1.1.1: icmp_seq=6 ttl=64 time=6.01 ms
64 bytes from 12.1.1.1: icmp_seq=7 ttl=64 time=5.62 ms
64 bytes from 12.1.1.1: icmp_seq=8 ttl=64 time=5.24 ms
64 bytes from 12.1.1.1: icmp_seq=9 ttl=64 time=5.17 ms
64 bytes from 12.1.1.1: icmp_seq=10 ttl=64 time=5.48 ms

--- 12.1.1.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9010ms
rtt min/avg/max/mdev = 4.566/5.518/6.584/0.543 ms
```

Figura 3.11. Prueba de conectividad mediante ping desde el nr-UE al UPF (12.1.1.1) a través de la interfaz oaitun_ue1.

```
danielcarballo@TFEDCR:~/TFG/evidencias$ cat ping_internet.log
PING 8.8.8.8 (8.8.8.8) from 12.1.1.100 oaitun_ue1: 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=7.17 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=6.61 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=6.88 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=6.28 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=115 time=7.53 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=115 time=5.26 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=115 time=6.60 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=115 time=6.48 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=115 time=6.22 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=115 time=7.56 ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9014ms
rtt min/avg/max/mdev = 5.256/6.657/7.562/0.650 ms
```

Figura 3.12. Prueba de conectividad mediante ping desde el nr-UE a una dirección externa (8.8.8.8), verificando la interfaz N6.

3.5.1. Configuración de tiempo real del sistema operativo

La documentación de OAI señala explícitamente que el procesamiento en tiempo real es un requisito crítico del rfsimulator (apartado 2.2.2.3), y en apoyo de la mitigación de la saturación del buffer de envío descrita en el apartado 3.4, se ha aplicado sobre la VM un conjunto de medidas orientadas a reducir la latencia y el jitter:

- Kernel de tiempo real (linux-image-rt-amd64): Si bien el kernel de Linux habitual de Debian funciona correctamente, esta versión del kernel convierte en expropiables (preemptibles) prácticamente todas las secciones críticas del kernel. Esto significa que prácticamente todas las rutinas internas pueden ser interrumpidas, lo que facilita el correcto funcionamiento del gNB al ser un proceso en tiempo real donde estos pequeños tiempos de espera entre rutina y rutina pueden marcar la diferencia. En resumen, el kernel RT reduce drásticamente esa latencia máxima de interrupción, a costa de un ligero descenso del rendimiento medio del sistema.
- Deshabilitación de estados de bajo consumo (P-states y C-states): los procesadores modernos entran en estados de bajo consumo (C-states) cuando un núcleo está inactivo, y ajustan su voltaje y frecuencia mediante estados de rendimiento (P-states) para ahorrar energía; abandonar estos estados para atender una nueva tarea introduce una latencia de varios microsegundos, suficiente para provocar la pérdida de un plazo de subtrama de 1 ms. Los parámetros de arranque del kernel intel_pstate=disable, processor.max_cstate=1 e idle=poll fuerzan a la CPU a permanecer en su estado de máximo rendimiento de forma constante, eliminando esta fuente de latencia variable.

Estas medidas reducen ligeramente la latencia del planificador del sistema invitado, pero no eliminan la fuente más importante del jitter: la planificación del propio hipervisor sobre un conjunto de vCPUs compartidas. El hecho de no poder acceder a los controles a nivel de nodo

de Proxmox es una restricción que plantea el entorno universitario empleado y causa que la mitigación final de este problema se tuviera que buscar desde el interior del sistema invitado, mediante el ajuste de los buffers de socket como se describe en el apartado 3.4.

3.6. Metodología de depuración empleada

A lo largo de todo el proceso de despliegue se ha seguido una metodología de depuración sistemática por capas, verificando el correcto funcionamiento de cada interfaz y socket antes de avanzar a la capa superior de la pila de protocolos. Esta forma de depurar es la natural cuando se analizan los logs pues, aunque no siempre son del todo descriptivos suelen hacer referencia al último protocolo de la pila en el que se han quedado.

Para analizar los protocolos como tal en los que había problemas se ha recurrido de forma recurrente a la inspección de bajo nivel a través del sistema de ficheros /proc (en particular /proc/net/sctp/eps y /proc/net/tcp) debido a que los contenedores son muy básicos y muchos no contienen acceso a herramientas más refinadas para depurar. Además, la verificación mediante ps aux de los argumentos de arranque también ha sido de ayuda, dado que los parámetros que pasamos a la ejecución a través de useAdditionalOptions prevalecen sobre los valores definidos en los ficheros de configuración estáticos, lo que podía ocultar la causa real de un fallo.

3.7. Guía de despliegue paso a paso

Reunidos ya el diagnóstico de incidencias (apartado 3.3) y el estado del sistema (apartado 3.5), cierra este capítulo el procedimiento completo, práctico y secuencial, seguido para levantar el clúster 5G descrito a lo largo de él, desde la preparación de la máquina virtual Debian hasta el despliegue final con kubectl. Su objetivo es que el despliegue sea reproducible por un tercero sin necesidad de reconstruir por sí mismo el proceso de prueba y error ya documentado en los apartados anteriores. Los comandos se presentan como referencia representativa del procedimiento seguido; las direcciones IP, nombres de interfaz y rutas concretas deben adaptarse a cada entorno.

3.7.1. Preparación de la máquina virtual Debian

El primer paso de todo el despliegue es la propia máquina virtual sobre la que se apoya el resto del sistema: si esta base no queda correctamente configurada, ningún ajuste posterior en Kubernetes o en los charts de Helm podrá compensarlo, especialmente en lo relativo a los requisitos de tiempo real descritos en el apartado 3.1.1.

3.7.1.1. Recursos asignados a la VM

Sobre el hipervisor Proxmox se crea una máquina virtual Debian con el mayor número de vCPUs y memoria que la infraestructura disponible permita asignar. Para este despliegue se utilizaron 18 vCPUs y 32 Gi de RAM pero con 6 vCPUs y 16 Gi de RAM deberían ser suficientes. Tanto el

5GC como el gNB y el nr-UE compiten por los mismos recursos de CPU dentro de un único nodo Kubernetes, debido a esto es aconsejable deshabilitar el balloon de memoria y, si el hipervisor lo permite, fijar el tipo de CPU expuesto a la VM como host, de forma que el sistema operativo invitado tenga visibilidad de las instrucciones y características reales del procesador físico.

3.7.1.2. Instalación del kernel de tiempo real

Una vez instalado el sistema base, se sustituye el kernel estándar por la variante de tiempo real, tal y como se justificó en el apartado 3.5.1:

```
sudo apt update
sudo apt install -y linux-image-rt-amd64 linux-headers-rt-amd64
sudo reboot
# Tras reiniciar, comprobar que el kernel activo es el RT:
uname -r
# debe mostrar algo similar a: 6.1.0-XX-rt-amd64
```

3.7.1.3. Configuración de GRUB: gobernador de CPU y estados C/P

Se edita la línea de arranque del kernel para deshabilitar los estados de bajo consumo:

```
sudo nano /etc/default/grub
# Modificar la línea GRUB_CMDLINE_LINUX_DEFAULT añadiendo:
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_pstate=disable processor.max_cstate=1
idle=poll"

sudo update-grub
sudo reboot
```

3.7.1.4. Instalación de Docker

Docker actúa como la base donde se ejecutan los contenedores tanto para Minikube (apartado 2.4.3) como para cada Pod que se despliega sobre el clúster. Además, Minikube debe tener acceso a Docker sin permisos de root:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
docker version
```

3.7.1.5. Instalación de kubectl, Minikube y Helm

```
# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

3.7.1.6. Ampliación de los buffers de entrada y salida para rfsim

```
sudo sh -c 'echo 67108864 > /proc/sys/net/core/wmem_max'

sudo sh -c 'echo 67108864 > /proc/sys/net/core/rmem_max'

sudo sh -c 'echo 16777216 > /proc/sys/net/core/wmem_default'

sudo sh -c 'echo 16777216 > /proc/sys/net/core/rmem_default'
```

3.7.2. Puesta en marcha de Minikube y Multus CNI

Con todo lo ejecutado hasta aquí la VM debería estar preparada. El siguiente paso es levantar el propio clúster de Kubernetes y dotarlo de la capacidad multi-interfaz que el 5GC necesita, siguiendo lo descrito en los apartados 2.4.3 y 2.5.

3.7.2.1. Arranque de Minikube con el driver Docker

```
minikube start --driver=docker --cpus=6 --memory=16g \
  --kubernetes-version=stable

# Comprobación del estado del nodo:
kubectl get nodes -o wide
minikube status
```

3.7.2.2. Instalación de Multus CNI

```
kubectl apply -f https://raw.githubusercontent.com/k8snetworkplumbingwg/multus-
cni/master/deployments/multus-daemonset-thick.yml

# Verificar que el DaemonSet de Multus está en ejecución:
kubectl get pods -n kube-system | grep multus
```

3.7.3. Configuración de los values.yaml del 5GC y la RAN

Con el clúster y la red multi-interfaz ya en pie, empezamos la configuración de los charts de Helm. Cada componente del sistema se instancia a partir de un chart genérico de OAI, cuyo comportamiento concreto se establece mediante la modificación del fichero values.yaml correspondiente y en casos específicos el config.yaml (apartado 2.4.4).

3.7.3.1. Obtención de los charts de OAI

```
git clone https://github.com/Mouradnetworking/OAI_5G_SA_Helmchart_kubernetes.git
cd OAI_5G_SA_Helmchart_kubernetes
```

El repositorio clonado con este comando descarga los charts que pertenecen a OAI en un formato ya preparado para ser desplegados usando Helm, lo que simplifica significativamente el proceso. Pero aún se deben modificar algunos valores para que el clúster funcione como deseamos.

3.7.3.2. values.yaml de las funciones de red del núcleo

En el values.yaml de cada NF que participe en una interfaz punto a punto (apartado 2.2.4) se declara la interfaz Multus a utilizar mediante una anotación de red y se fija su dirección IP, en lugar de dejar que el Pod dependa únicamente de su interfaz eth0 por defecto. Extracto representativo del values.yaml (primera figura) y del config.yaml (segunda figura) del AMF:

```
multus:
  ## If you don't want to add a default route in your pod then leave this field empty
  defaultGateway: "172.21.7.254"
  n2Interface:
    create: true
    Ipadd: "172.21.6.95"
    Netmask: "22"
    ## If you do not have a gateway leave the field empty
    Gateway:
    ## If you do not want to add any routes in your pod then leave this field empty
    routes: [{'dst': '10.8.0.0/24','gw': '172.21.7.254'}]
    hostInterface: "eth0" # Interface of the host machine on which this pod will be scheduled
```

Figura 3.13. Fragmento del values.yaml del AMF: bloque multus con la declaración de la interfaz n2, su dirección IP estática y la interfaz del anfitrión.

```
### Each NF takes its local SBI interfaces and remote interfaces from here, unless it gets them using NRF discov
ery mechanisms
nfs:
  amf:
    host: oai-amf
    sbi:
      port: 80
      api_version: v1
      interface_name: eth0
    n2:
      interface_name: n2
      port: 38412
```

Figura 3.14. Fragmento del config.yaml del AMF: bloque nfs con los parámetros de la interfaz SBI y de la interfaz n2 (puerto 38412).

El mismo principio se aplica al SMF (interfaz N4), al UPF (interfaces N3, N4 y N6) y al resto de funciones, adaptando el nombre de la interfaz y la dirección IP.

3.7.3.3. values.yaml del gNB y del nr-UE

En el gNB se configuran las interfaces N2 y N3 de forma análoga al resto de NFs, además de los parámetros propios de la celda y del enlace rfsimulator descritos en los apartados 2.2.2.1 y 3.4:

```
multus:
  # if default gatway is empty then it will be removed
  defaultGateway: ""
  n2Interface:
    create: true
    # #name inside the pod is hardcoded right now
    # name: "n2"
    IPadd: "172.21.6.94"
    Netmask: "22"
    # if gatway is empty then it will be removed
    #Gateway: "172.21.7.254"
    #routes: [{'dst': '10.8.0.0/24','gw': '172.21.7.254'}, {'dst': '10.9.0.0/24','gw': '172.21.7.254'}]
    hostInterface: "eth0"
  n3Interface:
    create: true
    IPadd: "172.21.8.91"
    # #name inside the pod is hardcoded right now
    # name: "n3"
    Netmask: "22"
    # if gatway is empty then it will be removed
    #Gateway: "172.21.11.254"
    #routes:
    hostInterface: "eth0"      # Interface of the host machine on which this pod will be scheduled
```

Figura 3.15. Fragmento del values.yaml del gNB: bloque multus con la declaración de las interfaces n2 y n3.

```
config:
  timeZone: "Europe/Paris"
  useAdditionalOptions: "--sa --continuous-tx --rfsim"
  gnbName: "FGE-USTHB"
  mcc: "001"    # check the information with AMF, SMF, UPF
  mnc: "01"     # check the information with AMF, SMF, UPF
  tac: "1"      # check the information with AMF
  sst: "1"  #currently only 4 standard values are allowed 1,2,3,4
  usrp: rfsim #allowed values rfsim, b2xx, n3xx or x3xx
  amfhost: "172.21.6.95"  # amf ip-address or service-name oai-amf-svc or 172.21.6.94
  n2IfName: "n2"    # if multus.n2Interface.create is true then use n2
  n3IfName: "n3"   #if multus.n3Interface.create is true then use n3 or you can only use 1 interface n2 or eth0
```

Figura 3.16. Fragmento del values.yaml del gNB: bloque config con los parámetros de celda, el modo rfsim y el parámetro amfhost.

Y en el nr-UE, el enlace rfsimulator apunta a la dirección macvlan del gNB:

```
multus:
  create: true
  ipadd: "172.21.6.27"  # subnet should be able to reach the gNB
  netmask: "22"
  # if default route is not needed then leave the field empty
  # defaultGateway: "172.21.7.254"
  hostInterface: "eth0"

config:
  timeZone: "Europe/Paris"
  rfSimServer: "172.21.6.94"    # ip-address of rfsim or service name oai-gnb or oai-du
  fullImsi: "001010000000100"       # make sure all the below entries are present in the subscriber database
  fullKey: "fec86ba6eb707ed08905757b1bb44b8f"
  opc: "C42449363BBAD02B66D16BC975D77CC1"
  dnn: "oai"
  sst: "1"                    # configure according to gnb and amf, smf and upf
  # sd: "16777215"
  usrp: "rfsim"            # allowed rfsim, b2xx, n3xx, x3xx
  useAdditionalOptions: "--sa -r 106 --numerology 1 -C 3619200000 --rfsim --nokrnmod"
```

Figura 3.17. Fragmento del values.yaml del nr-UE: interfaz multus y parámetro rfSimServer apuntando a la dirección macvlan del gNB.

3.7.4. Despliegue con Helm/kubectl y verificación de ConfigMaps

En el momento de la instalación Helm sustituye los valores del values.yaml en un ConfigMap que Kubernetes monta dentro del Pod correspondiente como fichero de configuración estático del proceso OAI (apartado 2.4.1.5).

Es recomendable comprobar los ConfigMap para comprobar que los cambios se han procesado correctamente puesto que hay varios tipos de errores (identación, o nombres de claves) que pueden generar un ConfigMap sintácticamente valido, pero funcionalmente incorrecto sin que Helm lo marque como error.

3.7.4.1. Orden de despliegue

En 5G el orden de instalación es muy importante, podemos tener bien toda la configuración que si no levantamos los Pods en el orden correcto lo más probable es que el UE no sea capaz de establecer la interfaz oaitun (la interfaz que da acceso al UE al resto de la red). El orden correcto a seguir es el siguiente:

1. La base de datos, en el caso de este proyecto basic-mysql.
2. El UDR y el NRF.
3. El resto de los Pods pertenecientes a 5GC (AMF, SMF, UPF, AUSF y UDM).
4. El gNB.

5. Y, por último, siendo especialmente relevante reiniciar este por en caso de haber reiniciado cualquier otro, el NR-UE.

A continuación, muestro como se levantarían de forma individual cada uno de estos Pods:

```
helm install oai-mysql ./mysql -f oai-mysql/values.yaml
helm install oai-udr   ./oai-udr   -f oai-udr/values.yaml
helm install oai-nrf   ./oai-nrf   -f oai-nrf/values.yaml
helm install oai-amf   ./oai-amf   -f oai-amf/values.yaml
helm install oai-smf   ./oai-smf   -f oai-smf/values.yaml
helm install oai-upf   ./oai-upf    -f oai-upf/values.yaml
helm install oai-ausf  ./oai-ausf   -f oai-ausf/values.yaml
helm install oai-udm   ./oai-udm   -f oai-udm/values.yaml

helm install oai-gnb   ./oai-gnb   -f oai-gnb/values.yaml
helm install oai-nrue  ./oai-nrue  -f oai-nrue/values.yaml
```

También es posible levantar los Pods del core de manera conjunta haciendo uso de la carpeta oai-5g-basic que tiene un Chart que se encarga de levantarlos en orden con HELM aunque si se utiliza esta opción se deberá tener en cuenta que el values.yaml existente en dicha carpeta sobrescribirá los values.yaml de las carpetas de las diferentes funciones, además hay cambios en diferentes funciones que se deberán seguir haciendo en las carpetas individuales de las funciones.

3.7.4.2. Inspección de los ConfigMaps generados

```
kubectl get configmap
kubectl describe configmap oai-amf-configmap
kubectl desribe configmap …
```

Una vez ejecutado los diferentes comandos de HELM podemos comprobar lo que hemos levantado en realidad mediante una comprobación de los ConfigMap utilizando los comandos de arriba. Esto nos permite comprobar principalmente si los cambios que hemos hecho en los values.yaml o en los config.yaml se han aplicado correctamente. En caso de que lo que aparezca al ejecutar el comando no sea lo esperado se puede modificar de manera no permanente mediante un kubectl edit.

3.7.4.3. Verificación del despliegue

Por último, se recorren las mismas comprobaciones de bajo nivel empleadas durante la depuración del capítulo 3 (apartado 3.6), en el mismo orden en que se fueron resolviendo las incidencias: creación de las interfaces macvlan, asociación SCTP, registro NGAP y, finalmente, conexión rfsimulator:

```
kubectl get pods -o wide

# Interfaces macvlan dentro del Pod del AMF:
kubectl exec deploy/oai-amf -- ip a

# Asociación SCTP N2 establecida:
kubectl exec deploy/oai-amf -- cat /proc/net/sctp/eps

# Conexión TCP de rfsimulator entre gNB y UE:
kubectl exec deploy/oai-gnb -- cat /proc/net/tcp
```

```
# Logs de sincronización del UE:
kubectl logs deploy/oai-nrue
```

Una vez que hemos completado todos los pasos mencionados el clúster debería quedar plenamente operativo:

1. El gNB debería estar registrado frente al AMF mediante NGAP sobre SCTP



Figura 3.18. Tabla de gNB conectados publicada por el AMF, con el gNB FGE-USTHB en estado Connected.

2. El nr-UE debería completar la sincronización de capa física y el registro NAS, y se establece una sesión PDU con conectividad de plano de usuario verificada, tal y como se describe en los apartados 3.4 y 3.5.

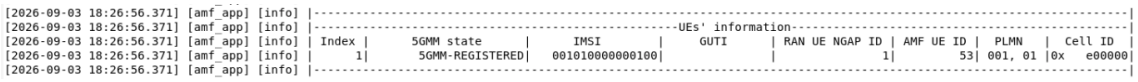


Figura 3.19. Tabla de UE registrados publicada por el AMF, con el equipo de usuario en estado 5GMM-REGISTERED.

3. Y al entrar en el Pod del UE deberíamos ser capaces de ver la interfaz oaitun_ue1 levantada, a través de la cual deberíamos poder hacer ping tanto al UPF (12.1.1.1) como a cualquier otro servicio externo como puede ser el DNS de Google (8.8.8.8).

4. Marco regulador

El despliegue de una red 5G, aunque sea en un laboratorio, se sitúa en un punto en el que se ve afectado por varios marcos normativos:

1. La estandarización técnica que define la interoperabilidad del sistema.
2. La regulación administrativa del espectro radioeléctrico.
3. Las licencias de software que condicionan el uso y la redistribución de los componentes empleados.
4. La legislación de protección de datos que aplica a los identificadores de suscriptor que la red maneja.

En este capítulo se analiza cada uno de estos aspectos y su influencia concreta sobre las decisiones de diseño adoptadas a lo largo del trabajo.

4.1. Estándares técnicos aplicables

La totalidad del sistema desplegado se rige por las especificaciones del 3GPP (3rd Generation Partnership Project), organismo que agrupa a las principales asociaciones de estandarización de telecomunicaciones y publica las especificaciones técnicas (TS) que definen el sistema 5G.

A diferencia de una norma jurídica, estas especificaciones no son de obligado cumplimiento por imperativo legal, pero son necesarias para la interoperabilidad, es decir, si un equipo no las sigue no podrá comunicarse con el resto del ecosistema.

Las especificaciones concretas que condicionan este trabajo son las siguientes:

- 3GPP TS 23.501 [1] y TS 23.502 [2] definen la arquitectura basada en servicios del núcleo 5G y los procedimientos entre funciones de red. La separación de funciones adoptada en el despliegue y la secuencia de establecimiento de sesión PDU descrita en el apartado 3.5 son una implementación directa de estas especificaciones.
- 3GPP TS 38.300 [3], TS 38.211 [5] y TS 38.401 [4] definen la red de acceso NG-RAN, los canales físicos y la modulación de New Radio. La parametrización de la celda documentada en el apartado 3.4 está enteramente determinada por estas especificaciones.
- 3GPP TS 38.413 [10] (NGAP), TS 29.281 [11] (GTP-U) y TS 29.244 [12] (PFCP) definen los protocolos de las interfaces N2, N3 y N4 respectivamente. La segmentación de red mediante Multus CNI descrita en el apartado 3.2 replica la separación de planos que estas especificaciones presuponen.
- 3GPP TS 24.501 [13] define el protocolo NAS, y TS 33.501 [14] la arquitectura de seguridad del sistema 5G, incluyendo el procedimiento de autenticación 5G-AKA y el mecanismo de ocultación del identificador permanente de suscriptor.

- ETSI GS NFV 002 [15] establece el marco arquitectónico de virtualización de funciones de red, y las especificaciones de la O-RAN Alliance definen las interfaces abiertas de la RAN desagregada descrita en el apartado 2.2.6.

4.2. Regulación del espectro radioeléctrico

En España, lugar donde se ha realizado este trabajo, el espectro radioeléctrico es un bien de dominio público cuya titularidad y administración corresponden al Estado, conforme al artículo 85 de la Ley 11/2022, de 28 de junio, General de Telecomunicaciones [38], que transpone la Directiva (UE) 2018/1972 por la que se establece el Código Europeo de las Comunicaciones Electrónicas. Su ordenación se concreta en el Cuadro Nacional de Atribución de Frecuencias (CNAF), aprobado por la Orden TDF/732/2026, de 10 de julio [39], que deroga la edición anterior de 2021 e incorpora las previsiones adoptadas en la Conferencia Mundial de Radiocomunicaciones de 2023.

La banda n78 empleada en la configuración de este despliegue (3,3–3,8 GHz) se corresponde en su mayor parte con la banda 3400-3800 MHz, regulada por la nota de utilización nacional UN-107 y armonizada a escala europea por la Decisión 2008/411/CE, modificada por la Decisión de Ejecución 2014/276/UE [40]. Los derechos de uso sobre esta banda se adjudicaron mediante procedimientos de licitación a operadores con título habilitante, con un límite de 120 MHz por operador o grupo empresarial en cualquier ámbito territorial, por lo que la emisión en dichas frecuencias sin autorización constituiría una infracción del régimen de uso del dominio público radioeléctrico regulado por el Real Decreto 123/2017, de 24 de febrero [41].

No obstante, el propio CNAF contempla una vía específica para usos como el que motivaría un laboratorio académico:

> *"el Secretario de Estado de Telecomunicaciones e Infraestructuras Digitales autorizará usos de carácter temporal o experimental distintos de los previstos por el Cuadro Nacional de Atribución de Frecuencias (CNAF), siempre que no causen perturbaciones a estaciones radioeléctricas autorizadas según la legislación vigente ni provoquen interferencias perjudiciales a otras administraciones"* [39]

Teniendo en cuenta esto, de haber utilizado un SDR como pudiera ser un USRP, que tendría que emitir en banda n78, hubiesen sido necesarias precauciones como un entorno de conducción por cable o una jaula de Faraday que garantice la ausencia de radiación al exterior.

La sustitución del equipamiento de radiofrecuencia por el módulo rfsimulator sitúa este trabajo fuera del ámbito de aplicación de esta normativa.

El módulo no genera señal radioeléctrica: sustituye la cadena de radiofrecuencia por el intercambio de muestras en banda base sobre un socket TCP. De este modo el clúster 5G levantado en este trabajo no genera emisión alguna.

Lo que en el apartado 2.2.2.3 se presentó como una solución ante la indisponibilidad de hardware SDR resulta ser, además, la única opción admisible desde el punto de vista regulatorio en ausencia de título habilitante o autorización experimental.

4.3. Seguridad de redes 5G

Las redes 5G están sujetas a un régimen específico de ciberseguridad derivado del reconocimiento de su condición de infraestructura crítica. A escala europea, la Comisión y el grupo de cooperación NIS elaboraron el denominado EU 5G Toolbox [45], un conjunto coordinado de medidas de mitigación de riesgos que incluye tanto aspectos técnicos como consideraciones sobre la diversificación de proveedores. En el ordenamiento español, el Real Decreto-ley 7/2022 [43], de 29 de marzo, establece los requisitos para garantizar la seguridad de las redes y servicios de comunicaciones electrónicas de quinta generación, e impone obligaciones de análisis de riesgos y de gestión de proveedores a operadores y suministradores. La Directiva (UE) 2022/2555 (NIS2) [44] refuerza adicionalmente las obligaciones de seguridad aplicables a los operadores de comunicaciones electrónicas.

Aunque un laboratorio académico aislado no queda sujeto a estas obligaciones, dos aspectos del despliegue guardan relación directa con sus objetivos.:

1. La propia motivación del enfoque abierto y desagregado descrito en el apartado 2.2.6 responde parcialmente a la preocupación regulatoria por la dependencia de un número reducido de proveedores dado que una RAN basada en interfaces abiertas facilita la diversificación que el EU 5G Toolbox [45] promueve.

2. La configuración de seguridad del enlace radio empleada en este trabajo (algoritmo de cifrado nea0) cumple su función para un despliegue en laboratorio donde no se usa información real, pero resultaría inadmisible en un despliegue real, donde la selección de algoritmos debe realizarse conforme a lo establecido en 3GPP TS 33.501 [14].

4.4. Licencias de software y propiedad intelectual

La totalidad de los componentes empleados en el despliegue son software libre o de código abierto o permisivas como es el caso de OAI, a continuación, expongo las diferentes licencias de uso:

Los componentes de OpenAirInterface se distribuyen bajo licencia Collaborative Standards Software License v1.0 (CSSL). Esta licencia para el ámbito académico expone lo siguiente:

"Con sujeción a los términos y condiciones de esta Licencia, el Licenciante y cada Colaborador conceden por la presente a Usted una licencia de patente perpetua, de alcance mundial, no exclusiva, gratuita, libre de regalías e irrevocable (salvo por lo dispuesto en esta sección) para fabricar, encargar la fabricación, usar y transferir de cualquier otro modo (excluida la venta) la Obra, únicamente con fines de estudio, prueba e investigación, aplicándose dicha licencia exclusivamente a aquellas reivindicaciones de patente susceptibles de ser licenciadas por el Licenciante o por dicho Colaborador que resulten necesariamente infringidas, respectivamente, por la Obra y/o por la Contribución o Contribuciones de dicho Colaborador, ya sea

aisladamente o en combinación con la Obra a la que tal Contribución fue enviada («Patentes Esenciales»).”[42]

Kubernetes, Helm, Docker y Multus CNI se distribuyen bajo Apache 2.0, y el sistema operativo Debian bajo un conjunto de licencias conformes con las Debian Free Software Guidelines.

4.5. Protección de datos y privacidad del suscriptor

Los sistemas 5G reales manejan datos de carácter personal según el Reglamento (UE) 2016/679 [46] (RGPD) y de la Ley Orgánica 3/2018 [47], de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.

El SUPI (Subscription Permanent Identifier) identifica de manera unívoca a un abonado, y con él se podría hacer un seguimiento tanto de la localización del usuario en la red como de sus acciones en esta. Además del SUPI, otros datos como los identificadores de sesión, las direcciones IP asignadas y los registros de señalización almacenados por las funciones de red son también información sensible que se podría llegar a asociar a una persona.

En el despliegue realizado no se ha utilizado ningún dato personal real. Los identificadores empleados no corresponden a ningún usuario real y fueron creados únicamente para uso en laboratorio. Esta circunstancia sitúa el despliegue fuera del ámbito de aplicación material del RGPD. Aun así, considero que este apartado es importante a nivel teórico pues muestra una de las principales regulaciones que afecta al ámbito de las telecomunicaciones y es especialmente relevante a nivel laboral donde estas restricciones sí que se aplican.

5. Entorno socioeconómico

Este capítulo presenta el presupuesto asociado a la realización del trabajo, analiza el impacto socioeconómico esperado de sus resultados y examina su contribución a los Objetivos de Desarrollo Sostenible de la Agenda 2030.

5.1. Presupuesto del proyecto

Al igual que todo proyecto, este Trabajo de Fin de Grado ha supuesto un coste económico, en este apartado se recopila el presupuesto estimado para la ejecución del proyecto teniendo en cuenta los costes de personal, la amortización del equipamiento empleado y las licencias de software necesarias, además también se tienen en cuenta costes indirectos e impuestos. La duración estimada del trabajo ha sido de cuatro meses con aproximadamente 300 horas dedicadas por parte del autor.

En cuanto al equipamiento, el servidor de Proxmox donde se ejecuta la máquina virtual donde se ha levantado el clúster pertenece al departamento de Telemática de la universidad Carlos III de Madrid, por lo que para estimar el coste he consultado el precio de alquilar una máquina virtual con las prestaciones mínimas para que se pueda ejecutar el clúster durante los cuatro meses que duró el desarrollo. Se considera asimismo la amortización del equipo portátil empleado como estación de trabajo para el acceso remoto al servidor, aplicando un criterio de amortización lineal a sesenta meses [37].

Concepto	Cantidad	Coste unitario	Importe
Costes de personal			
Ingeniero junior (autor)	300 h	25€/h	7.500,00 €
Tutor académico (supervisión)	25 h	60€/h	1.500,00 €
Amortización de equipamiento			
Servidor Proxmox (4.000 €, 20 % dedicación)	4 meses	13€/mes	52 €
Equipo portátil de desarrollo (1.200 €)	4 meses	20€/mes	80,00 €
Licencias de software			
OpenAirInterface, Kubernetes, Helm, Docker, Debian	—	0 €	0,00 €
Microsoft 365 (licencia institucional UC3M)	4 meses	0 €	0,00 €
Subtotal			**9.132 €**
Costes indirectos (15 %)			1.369.8 €
Base imponible			**10.501,8 €**
IVA (21 %)			2.205,38 €
TOTAL			**12.707,18 €**

Tabla 5.1. Presupuesto detallado del proyecto.

El coste total asciende a doce mil setecientos siete euros con diecinueve céntimos. El coste total de este trabajo es bastante bajo gracias a varios motivos entre los que destaco el uso de rfsim

que supuso un ahorro significativo al no tener que utilizar un SDR y el hecho de que todo el software utilizado fuera opensource, a excepción de Microsoft 365 que gracias a la licencia institucional no costó dinero, lo que también redujo el coste significativamente.

5.2. Impacto socioeconómico

Este trabajo consiste en un laboratorio reproducible y la documentación del proceso. Teniendo esto en cuenta, su impacto socioeconómico resulta principalmente en una reducción de las barreras de acceso.

La barrera de entrada a la experimentación con tecnología 5G ha sido tradicionalmente elevada. Un banco de pruebas basado en equipamiento comercial requiere licencias de software propietario, hardware de radiofrecuencia específico y, en el caso de emitir en bandas licenciadas, un título habilitante para el uso del espectro como se ha definido en el marco regulador. En esta memoria se demuestra que es posible obtener una red 5G Standalone funcional de extremo a extremo sobre una única máquina virtual, sin coste de licencia y sin emisión radioeléctrica. Además, el coste de replicar el entorno para un estudiante es muy bajo, siendo la mayor parte del coste el acceso a una máquina con los suficientes recursos.

A nivel económico este trabajo no tiene un impacto relevante. El clúster levantado, aun siendo funcional, no ha sido probado con radiofrecuencia real y los problemas causados por haberlo levantado en una máquina virtual sin permisos de nodo, sospecho que se acentuarían en un entorno real. Con todo esto queda descartado el uso de este proyecto con fines económicos.

5.3. Contribución a los Objetivos de Desarrollo Sostenible

El trabajo guarda relación con varios de los Objetivos de Desarrollo Sostenible aprobados por Naciones Unidas en la Agenda 2030, principalmente de forma indirecta, a través de la capacitación y de las propiedades del enfoque tecnológico adoptado.

- ODS 4 (Educación de calidad). Dado que el resultado principal del trabajo es un laboratorio reproducible acompañado de una guía de despliegue detallada, concebido como material docente, y teniendo en cuenta la meta 4.4, relativa al aumento del número de personas con las competencias técnicas necesarias para el empleo, se aprecia una relación directa entre este TFG y el ODS 4.
- ODS 12 (Producción y consumo responsables). Gracias a la virtualización se puede aprovechar de manera mucho más eficiente el hardware. Esto está relacionado con el consumo responsable, pues reduce la necesidad de utilizar más equipamiento y, por tanto, de generar más residuos electrónicos. Además, en el caso específico de este trabajo, el uso de rfsimulator en vez de un SDR evita por completo el equipamiento de radiofrecuencia específico, por lo que el impacto ecológico es menor aún.
- ODS 10 (Reducción de las desigualdades). Como ya se ha mencionado en el apartado 5.2, uno de los mayores impactos que tiene este trabajo es el de democratizar el acceso a la experimentación con tecnología 5G mediante software libre y hardware de propósito general. El uso de elementos relativamente sencillos de adquirir reduce la brecha entre instituciones con capacidad de inversión en equipamiento especializado y aquellas que no la tienen, tanto en el ámbito académico como entre regiones con distinto nivel de desarrollo.

Es importante mencionar que, si bien la virtualización tiene aspectos positivos relacionados con el ODS 12, en el caso específico de este trabajo la configuración que se lleva a cabo para intentar reducir el jitter conlleva un sobrecoste de procesamiento en comparación con una implementación sobre hardware dedicado, consumiendo más energía a cambio de reducir la latencia. Esto no invalida lo dicho sobre el ODS 12, pero sí lo matiza: para un despliegue tan pequeño como el que se ha realizado estos costes son mínimos, aunque es importante tenerlo en cuenta en caso de querer levantar un clúster de mayor tamaño de esta manera.

6. Conclusiones

6.1. Resumen del trabajo realizado

En este trabajo de Fin de Grado se ha abordado tanto el diseño como el despliegue de una red 5G Standalone completamente funcional, con la parte del core y de la red de acceso, sobre un clúster de Kubernetes levantado en un único nodo y empleando las NFs de código abierto proporcionadas por OpenAirInterface.

Se ha establecido un marco teórico que fundamenta las decisiones de diseño elegidas en este proyecto y sobre esta base teórica se documentó el despliegue de la red 5G, abarcando desde cómo se preparó el entorno y las incidencias hasta que se establecieron las sesiones de manera correcta y se pudo validar el plano de usuario mediante tráfico ICMP.

Así se ha obtenido una red 5G Standalone operativa de extremo a extremo en un único nodo de Kubernetes, con un núcleo de red plenamente funcional, un gNB que se registra correctamente en el AMF y un equipo de usuario que completa el registro NAS y establece una sesión PDU, permitiendo así al UE cursar tráfico de datos.

Hay que destacar que, además del resultado funcional, este trabajo aporta una caracterización del problema que supone ejecutar una red 5G en una máquina virtual sin control sobre el nodo, la pérdida de muestras en la conexión que simula el enlace aéreo, y la demostración de que puede mitigarse sin necesidad de acceso privilegiado al anfitrión.

6.2. Evaluación de los objetivos

A continuación, se evalúa, uno por uno, el grado de cumplimiento de los objetivos específicos planteados en el apartado 1.2.2, conforme a los criterios de aceptación allí definidos.

OE1 (desplegar el 5GC)	**Cumplido ✓**
Las ocho funciones de red del núcleo alcanzan el estado Running y se registran correctamente contra el NRF, tal y como se describe en el apartado 3.3	
OE2 (segmentación de red multi-interfaz)	**Cumplido ✓**
Tras resolver la incidencia de direccionamiento duplicado del apartado 3.3.1. Cada NAD crea su interfaz macvlan de forma correcta y verificable	
OE3 (despliegue de gNB y nr-UE con rfsimulator)	**Cumplido ✓**
La conexión TCP de rfsimulator se establece correctamente entre ambos procesos, tal y como se verificó en el apartado 3.4.	
OE4 (diagnóstico y resolución de incidencias)	**Cumplido ✓**
Las tres incidencias del núcleo (apartado 3.3) se diagnosticaron y resolvieron por completo, al igual que la saturación del buffer de envío del rfsimulator (apartado 3.4)	

y las incidencias de selección de slice y de política PCC surgidas en el establecimiento de la sesión PDU (apartado 3.5)	
OE5 (registro completo end-to-end)	**Cumplido ✓**
El UE completa la sincronización de capa física, la decodificación del MIB, el establecimiento de la conexión RRC, el registro NAS con autenticación 5G-AKA y el establecimiento de una sesión PDU, obteniendo direccionamiento IP sobre oaitun_ue1 y conectividad de datos verificada contra el UPF (apartado 3.5)	
OE6 (memoria autocontenida)	**Cumplido ✓**
El presente documento incluye marco teórico, implementación, guía de reproducción y esta evaluación de conclusiones	

Tabla 6.1. Evaluación del grado de cumplimiento de los objetivos específicos.

En conclusión, se han cumplido los seis objetivos específicos originales pues el trabajo entrega un despliegue funcional y reproducible junto con la documentación detallada del proceso de diagnóstico que condujo hasta él.

6.3. Líneas futuras de trabajo

A la hora de acotar este trabajo decidí centrarme en entender bien cómo interactúan las funciones de red básicas de una red 5G Standalone, dejando fuera otras cuestiones que también me habrían interesado. En este apartado recojo las que considero continuaciones más naturales.

1. **Integración con NFs de otros proyectos opensource**

   La primera evolución interesante de este trabajo sería desplegar funciones de red de proyectos o fabricantes distintos. Dado que todos requieren su ajuste a las especificaciones del 3GPP, un despliegue heterogéneo permitiría comprobar hasta qué punto se cumplen esas especificaciones y qué nivel de interoperabilidad se alcanza en la práctica. Está además muy relacionada con la arquitectura O-RAN que se describe en el apartado 2.2.6.

2. **Integración USRP**

   Otro punto a contemplar sería incorporar un USRP real, lo que permitiría contrastar los resultados obtenidos con rfsimulator frente a un enlace de radiofrecuencia de verdad y ver hasta qué punto la simulación es representativa. Eso sí, antes habría que resolver los requisitos regulatorios de uso del espectro que se comentan en el apartado 4.2.

3. **Cluster multinodo**

   Abriría la puerta a pruebas de carga con las que evaluar la solidez de la interacción entre Kubernetes y las funciones de red de OAI, y a estudiar el comportamiento del sistema bajo escalado horizontal y alta disponibilidad. Me parece la más relevante a nivel profesional, porque si el sector está adoptando estas plataformas de orquestación es precisamente por su capacidad de adaptarse a la carga de tráfico.

En resumen, como ya se indicaba en el alcance inicial del trabajo, el objetivo a cubrir es concreto y centrado en la certificación funcional de la solución propuesta, pero las posibilidades de evolucionar este ejercicio son múltiples con el objetivo de obtener una solución más eficiente y escalable.

Referencias

Especificaciones 3GPP y ETSI

[1] 3GPP, «System architecture for the 5G System (5GS); Stage 2», 3GPP TS 23.501, V19.8.0, Release 19. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144

[2] 3GPP, «Procedures for the 5G System (5GS); Stage 2», 3GPP TS 23.502, V19.8.0, Release 19. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3145

[3] 3GPP, «NR; NR and NG-RAN Overall description; Stage 2», 3GPP TS 38.300, V18.10.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3191

[4] 3GPP, «NG-RAN; Architecture description», 3GPP TS 38.401, V18.7.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3219

[5] 3GPP, «NR; Physical channels and modulation», 3GPP TS 38.211, V18.9.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3213

[6] 3GPP, «NR; Multiplexing and channel coding», 3GPP TS 38.212, V18.8.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3214

[7] 3GPP, «NR; Physical layer procedures for control», 3GPP TS 38.213, V18.8.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3215

[8] 3GPP, «NR; Radio Resource Control (RRC) protocol specification», 3GPP TS 38.331, V18.10.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3197

[9] 3GPP, «NR; Base Station (BS) radio transmission and reception», 3GPP TS 38.104, V18.14.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3202

[10] 3GPP, «NG-RAN; NG Application Protocol (NGAP)», 3GPP TS 38.413, V18.10.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3223

[11] 3GPP, «General Packet Radio System (GPRS) Tunnelling Protocol User Plane (GTPv1-U)», 3GPP TS 29.281, V18.4.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=1699

[12] 3GPP, «Interface between the Control Plane and the User Plane Nodes», 3GPP TS 29.244, V18.10.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3111

[13] 3GPP, «Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3», 3GPP TS 24.501, V18.13.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3370

[14] 3GPP, «Security architecture and procedures for 5G System», 3GPP TS 33.501, V18.12.0, Release 18. [En línea]. Disponible en: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3169

[15] ETSI, «Network Functions Virtualisation (NFV); Architectural Framework», ETSI GS NFV 002, V1.2.1, dic. 2014. [En línea]. Disponible en: https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/

Kubernetes, Helm y redes de contenedores

[16] The Kubernetes Authors, «Kubernetes Documentation — Concepts». [En línea]. Disponible en: https://kubernetes.io/docs/concepts/ [Accedido: 3-sep-2026]

[17] The Kubernetes Authors, «Service», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/services-networking/service/ [Accedido: 3-sep-2026]

[18] The Helm Authors, «Helm Documentation». [En línea]. Disponible en: https://helm.sh/docs/ [Accedido: 3-sep-2026]

[19] Kubernetes Network Plumbing Working Group, «Multus-CNI: a CNI meta-plugin for multi-homed pods in Kubernetes». [En línea]. Disponible en: https://github.com/k8snetworkplumbingwg/multus-cni [Accedido: 3-sep-2026]

[20] Kubernetes SIGs, «minikube Documentation». [En línea]. Disponible en: https://minikube.sigs.k8s.io/docs/ [Accedido: 3-sep-2026]

[21] The Kubernetes Authors, «Pods», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/workloads/pods/ [Accedido: 3-sep-2026]

[22] The Kubernetes Authors, «Deployments», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ [Accedido: 3-sep-2026]

[23] The Kubernetes Authors, «Persistent Volumes», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/storage/persistent-volumes/ [Accedido: 3-sep-2026]

[24] The Kubernetes Authors, «ConfigMaps», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/configuration/configmap/ [Accedido: 3-sep-2026]

[25] The Kubernetes Authors, «Secrets», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/configuration/secret/ [Accedido: 3-sep-2026]

[26] The Kubernetes Authors, «Namespaces», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ [Accedido: 3-sep-2026]

[27] The Kubernetes Authors, «Command line tool (kubectl)», Kubernetes Documentation. [En línea]. Disponible en: https://kubernetes.io/docs/reference/kubectl/ [Accedido: 3-sep-2026]

OpenAirInterface

[28] OpenAirInterface Software Alliance, «OpenAirInterface 5G Core Network Project». [En línea]. Disponible en: https://github.com/openairinterface/oai-cn5g-fed [Accedido: 3-sep-2026]

[29] OpenAirInterface Software Alliance, « OAI_5G_SA_Helmchart_kubernetes ». [En línea]. Disponible en: https://github.com/Mouradnetworking/OAI_5G_SA_Helmchart_kubernetes.git Accedido: 3-sep-2026]

[30] OpenAirInterface Software Alliance, «rfsimulator — RF simulation documentation». [En línea]. Disponible en: https://github.com/OPENAIRINTERFACE/openairinterface5g/blob/develop/radio/rfsimulator/README.md [Accedido: 3-sep-2026]

Trabajos relacionados

[31] C. Adigun, «Integrating Disaggregated OpenAirInterface O-RAN Components with Open5GS 5G Core», Medium, 2024. [En línea]. [Accedido: 3-sep-2026] Disponible en: https://futuredon.medium.com/integrating-disaggregated-openairinterface-o-ran-components-with-open5gs-5g-core-76e5deac1730

[32] OpenRAN Gym, «Deploying OpenAirInterface (OAI) 5G RAN and Core-network in Colosseum». [En línea]. Disponible en: https://openrangym.com/tutorials/oai-tutorial [Accedido: 3-sep-2026]

O-RAN

[33] O-RAN Alliance, «O-RAN: Towards an Open and Smart RAN», white paper. [En línea]. Disponible en: https://www.o-ran.org/ [Accedido: 3-sep-2026]

[34] O-RAN Software Community, «O-RAN Architecture Overview». [En línea]. Disponible en: https://docs.o-ran-sc.org/en/latest/architecture/architecture.html [Accedido: 3-sep-2026]

[35] M. Polese, L. Bonati, S. D'Oro, S. Basagni y T. Melodia, «Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges», IEEE Communications Surveys & Tutorials, vol. 25, n.º 2, pp. 1376-1411, 2023. arXiv:2202.01032 doi: 10.1109/COMST.2023.3239220

Industria, adopción y presupuesto

[36] GSMA, «5G Standalone: Full Steam Ahead?». [En línea]. Disponible en: https://www.gsma.com/solutions-and-impact/technologies/networks/gsma_resources/5g-standalone-full-steam-ahead/ [Accedido: 3-sep-2026]

[37] STRATO, «Alquiler barato de V-Server Linux», strato.es. [En línea]. Disponible en: https://www.strato.es/servidores/vps-linux/ [Accedido: 3-sep-2026]

Marco regulador

[38] España, «Ley 11/2022, de 28 de junio, General de Telecomunicaciones», Boletín Oficial del Estado, núm. 155, 29 jun. 2022. [En línea]. Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2022-10757 [Accedido: 4-sep-2026]

[39] España, «Orden TDF/732/2026, de 10 de julio, por la que se aprueba el Cuadro Nacional de Atribución de Frecuencias», Boletín Oficial del Estado, núm. 173, 17 jul. 2026. [En línea]. Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2026-15661 [Accedido: 4-sep-2026]

[40] Comisión Europea, «Decisión de Ejecución 2014/276/UE, de 2 de mayo de 2014, por la que se modifica la Decisión 2008/411/CE relativa a la armonización de la banda de frecuencias de 3400-3800 MHz para sistemas terrenales capaces de prestar servicios de comunicaciones electrónicas en la Comunidad», Diario Oficial de la Unión Europea, L 139, 14 may. 2014. [En línea]. Disponible en: https://eur-lex.europa.eu/legal-content/es/ALL/?uri=CELEX%3A32014D0276&print=true

[41] España, «Real Decreto 123/2017, de 24 de febrero, por el que se aprueba el Reglamento sobre el uso del dominio público radioeléctrico», Boletín Oficial del Estado, núm. 53, 3 mar. 2017. [En línea]. Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2017-2178 [Accedido: 4-sep-2026]

[42] OpenAirInterface Software Alliance, «Collaborative Standards Software License v1.0 (OAI CSSL)». [En línea]. Disponible en: https://openairinterface.org/oai-cssl/ [Accedido: 6-sep-2026]

[43] Real Decreto-ley 7/2022, de 29 de marzo, sobre requisitos para garantizar la seguridad de las redes y servicios de comunicaciones electrónicas de quinta generación. [En línea]. Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2022-4973 [Accedido: 7-sep-2026]

[44] Directiva (UE) 2022/2555 del Parlamento Europeo y del Consejo, de 14 de diciembre de 2022, relativa a las medidas destinadas a garantizar un elevado nivel común de ciberseguridad en toda la Unión, por la que se modifican el Reglamento (UE) n.º 910/2014 y la Directiva (UE) 2018/1972 y por la que se deroga la Directiva (UE) 2016/1148 (Directiva SRI 2). Diario Oficial de la Unión Europea, L 333, pp. 80-152, 27 dic. 2022. [En línea]. Disponible en: https://www.boe.es/buscar/doc.php?id=DOUE-L-2022-81963 [Accedido: 7-sep-2026]

[45] Comisión Europea, «The EU toolbox for 5G security», 29 ene. 2020. [En línea]. Disponible en: https://digital-strategy.ec.europa.eu/en/library/eu-toolbox-5g-security [Accedido: 7-sep-2026]

[46] Unión Europea, «Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos)», Diario Oficial de la Unión Europea, L 119, 4 may. 2016. [En línea]. Disponible en: https://eur-lex.europa.eu/eli/reg/2016/679/oj [Accedido: 7-sep-2026]

[47] España, «Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales», Boletín Oficial del Estado, núm. 294, 6 dic. 2018. [En línea]. Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673 [Accedido: 7-sep-2026]

Figuras

[48] Proyecto 5G, Escuela Politécnica Nacional, “Stack de protocolos,” Proyecto 5G. [En línea]. Disponible en: https://proyecto5g.epn.edu.ec/index.php?option=com_content&view=article&id=108:stack-de-protocolos&catid=2:uncategorised [Accedido: 7-sep-2026].

[49] D. Singh, “How to use multiple CNI plugin (Multus CNI),” Medium, [En línea]. Disponible en: https://medium.com/@dhilipsingh92/how-to-use-multiple-cniplugin-multus-cni-61d60711d849 [Accedido: 7-sep-2026].

[50] Red Hat, “Arquitectura de Kubernetes,” *Red Hat*, [En línea]. Disponible en: https://www.redhat.com/es/topics/containers/kubernetes-architecture. [Accedido: 7-sep-2026].

Anexo 1 – Configuración del Cluster

Interfaces de los Pods levantados:

```
===== pod/oai-amf-56946b77bc-c8dbl =====
Defaulted container "tcpdump" out of: tcpdump, amf, init (init)
eth0      Link encap:Ethernet  HWaddr 66:78:B6:A2:DD:01
          inet addr:10.244.0.187  Bcast:10.244.255.255  Mask:255.255.0.0
          inet6 addr: fe80::6478:b6ff:fea2:dd01/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:286388 errors:0 dropped:0 overruns:0 frame:0
          TX packets:364839 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:27818705 (26.5 MiB)  TX bytes:32003552 (30.5 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

n2        Link encap:Ethernet  HWaddr DE:4B:0F:E5:C5:4E
          inet addr:172.21.6.95  Bcast:172.21.7.255  Mask:255.255.252.0
          inet6 addr: fe80::dc4b:fff:fee5:c54e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:23922 errors:0 dropped:0 overruns:0 frame:0
          TX packets:23996 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2108110 (2.0 MiB)  TX bytes:2112188 (2.0 MiB)

 ===== pod/oai-ausf-6bc47c8f84-2k5c2 =====
 Defaulted container "tcpdump" out of: tcpdump, ausf, init (init)
 eth0      Link encap:Ethernet  HWaddr 02:9F:20:1E:19:C2
           inet addr:10.244.0.154  Bcast:10.244.255.255  Mask:255.255.0.0
           inet6 addr: fe80::9f:20ff:fe1e:19c2/64 Scope:Link
           UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
           RX packets:353417 errors:0 dropped:0 overruns:0 frame:0
           TX packets:449169 errors:0 dropped:0 overruns:0 carrier:0
           collisions:0 txqueuelen:1000
           RX bytes:34308693 (32.7 MiB)  TX bytes:39398466 (37.5 MiB)

 lo        Link encap:Local Loopback
           inet addr:127.0.0.1  Mask:255.0.0.0
           inet6 addr: ::1/128 Scope:Host
           UP LOOPBACK RUNNING  MTU:65536  Metric:1
           RX packets:0 errors:0 dropped:0 overruns:0 frame:0
           TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
           collisions:0 txqueuelen:1000
           RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
    ===== pod/oai-gnb-86dd67f548-jm6zf =====
    eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
            inet 10.244.0.188  netmask 255.255.0.0  broadcast 10.244.255.255
            inet6 fe80::c039:13ff:fe6a:701d  prefixlen 64  scopeid 0x20<link>
            ether c2:39:13:6a:70:1d  txqueuelen 1000  (Ethernet)
            RX packets 927  bytes 64525 (64.5 KB)
            RX errors 0  dropped 0  overruns 0  frame 0
            TX packets 92  bytes 6450 (6.4 KB)
            TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

    lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
            inet 127.0.0.1  netmask 255.0.0.0
            inet6 ::1  prefixlen 128  scopeid 0x10<host>
            loop  txqueuelen 1000  (Local Loopback)
            RX packets 0  bytes 0 (0.0 B)
            RX errors 0  dropped 0  overruns 0  frame 0
            TX packets 0  bytes 0 (0.0 B)
            TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

    n2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
            inet 172.21.6.94  netmask 255.255.252.0  broadcast 172.21.7.255
            inet6 fe80::40f8:5cff:fed6:51f8  prefixlen 64  scopeid 0x20<link>
            ether 42:f8:5c:d6:51:f8  txqueuelen 1000  (Ethernet)
            RX packets 23916  bytes 2106604 (2.1 MB)
            RX errors 0  dropped 0  overruns 0  frame 0
            TX packets 7978733677  bytes 210189793385086 (210.1 TB)
            TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

    n3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
            inet 172.21.8.91  netmask 255.255.252.0  broadcast 172.21.11.255
            inet6 fe80::d4fa:25ff:fe69:f478  prefixlen 64  scopeid 0x20<link>
            ether d6:fa:25:69:f4:78  txqueuelen 1000  (Ethernet)
            RX packets 7569292005  bytes 210162746245348 (210.1 TB)
            RX errors 0  dropped 0  overruns 0  frame 0
            TX packets 812  bytes 66088 (66.0 KB)
            TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

    ru: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
            inet 192.168.80.90  netmask 255.255.255.0  broadcast 192.168.80.255
            inet6 fe80::a8d0:faff:fe08:9e6d  prefixlen 64  scopeid 0x20<link>
            ether aa:d0:fa:08:9e:6d  txqueuelen 1000  (Ethernet)
            RX packets 13  bytes 948 (948.0 B)
            RX errors 0  dropped 0  overruns 0  frame 0
            TX packets 93  bytes 6538 (6.5 KB)
            TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
===== pod/oai-nr-ue-668664b8c-g7fpc =====
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.244.0.190  netmask 255.255.0.0  broadcast 10.244.255.255
        inet6 fe80::bc30:ebff:fe12:8067  prefixlen 64  scopeid 0x20<link>
        ether be:30:eb:12:80:67  txqueuelen 1000  (Ethernet)
        RX packets 1011  bytes 71681 (71.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 213  bytes 13196 (13.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

net1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.21.6.27  netmask 255.255.252.0  broadcast 172.21.7.255
        inet6 fe80::e488:d7ff:feec:2090  prefixlen 64  scopeid 0x20<link>
        ether e6:88:d7:ec:20:90  txqueuelen 1000  (Ethernet)
        RX packets 7954771482  bytes 209556901532888 (209.5 TB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 7546389199  bytes 209529924782886 (209.5 TB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

oaitun_ue1: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST>  mtu 1500
        inet 12.1.1.100  netmask 255.255.255.0  destination 12.1.1.100
        inet6 fe80::875d:5d28:4113:c057  prefixlen 64  scopeid 0x20<link>
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00  txqueuelen 500  (UNSPEC)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 11  bytes 528 (528.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

===== pod/oai-nrf-fdcd9dfc4-sxt4v =====
Defaulted container "tcpdump" out of: tcpdump, nrf
eth0      Link encap:Ethernet  HWaddr 32:1E:7B:F7:00:6F
          inet addr:10.244.0.159  Bcast:10.244.255.255  Mask:255.255.0.0
          inet6 addr: fe80::301e:7bff:fef7:6f/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1971951 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1438408 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:174773511 (166.6 MiB)  TX bytes:120416626 (114.8 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:160 (160.0 B)  TX bytes:160 (160.0 B)


        ===== pod/oai-udm-74978686bc-fqcbm =====
        Defaulted container "tcpdump" out of: tcpdump, udm, init (init)
        eth0      Link encap:Ethernet  HWaddr 9E:DC:19:FF:71:C0
                  inet addr:10.244.0.160  Bcast:10.244.255.255  Mask:255.255.0.0
                  inet6 addr: fe80::9cdc:19ff:feff:71c0/64 Scope:Link
                  UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
                  RX packets:355043 errors:0 dropped:0 overruns:0 frame:0
                  TX packets:451193 errors:0 dropped:0 overruns:0 carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:34485246 (32.8 MiB)  TX bytes:39581244 (37.7 MiB)

        lo        Link encap:Local Loopback
                  inet addr:127.0.0.1  Mask:255.0.0.0
                  inet6 addr: ::1/128 Scope:Host
                  UP LOOPBACK RUNNING  MTU:65536  Metric:1
                  RX packets:0 errors:0 dropped:0 overruns:0 frame:0
                  TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

    ===== pod/oai-udr-68d69698db-zm49s =====
    Defaulted container "tcpdump" out of: tcpdump, udr, init (init)
    eth0      Link encap:Ethernet  HWaddr 22:90:4C:F8:54:9F
              inet addr:10.244.0.162  Bcast:10.244.255.255  Mask:255.255.0.0
              inet6 addr: fe80::2090:4cff:fef8:549f/64 Scope:Link
              UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
              RX packets:356145 errors:0 dropped:0 overruns:0 frame:0
              TX packets:452122 errors:0 dropped:0 overruns:0 carrier:0
              collisions:0 txqueuelen:1000
              RX bytes:34928011 (33.3 MiB)  TX bytes:39676507 (37.8 MiB)

    lo        Link encap:Local Loopback
              inet addr:127.0.0.1  Mask:255.0.0.0
              inet6 addr: ::1/128 Scope:Host
              UP LOOPBACK RUNNING  MTU:65536  Metric:1
              RX packets:0 errors:0 dropped:0 overruns:0 frame:0
              TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
              collisions:0 txqueuelen:1000
              RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

```
===== pod/oai-upf-76f67878f6-7cmk5 =====
Defaulted container "tcpdump" out of: tcpdump, upf, init (init)
eth0      Link encap:Ethernet  HWaddr BA:CB:1F:68:0C:74
          inet addr:10.244.0.170  Bcast:10.244.255.255  Mask:255.255.0.0
          inet6 addr: fe80::b8cb:1fff:fe68:c74/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:329473 errors:0 dropped:0 overruns:0 frame:0
          TX packets:433311 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:32173401 (30.6 MiB)  TX bytes:37866244 (36.1 MiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

n3        Link encap:Ethernet  HWaddr 6E:5C:EC:82:8E:D8
          inet addr:172.21.8.95  Bcast:172.21.11.255  Mask:255.255.252.0
          inet6 addr: fe80::6c5c:ecff:fe82:8ed8/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:804 errors:0 dropped:0 overruns:0 frame:0
          TX packets:451 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:65336 (63.8 KiB)  TX bytes:27666 (27.0 KiB)

n4        Link encap:Ethernet  HWaddr 02:92:C8:DA:62:E3
          inet addr:192.168.24.2  Bcast:192.168.24.255  Mask:255.255.255.0
          inet6 addr: fe80::92:c8ff:feda:62e3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:48375 errors:0 dropped:0 overruns:0 frame:0
          TX packets:48382 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2557140 (2.4 MiB)  TX bytes:2552179 (2.4 MiB)

n6        Link encap:Ethernet  HWaddr AA:F6:AB:53:0A:FD
          inet addr:192.168.22.2  Bcast:192.168.22.255  Mask:255.255.255.0
          inet6 addr: fe80::a8f6:abff:fe53:afd/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:296 errors:0 dropped:0 overruns:0 frame:0
          TX packets:110 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:28020 (27.3 KiB)  TX bytes:7588 (7.4 KiB)
```

Figura A.1. Interfaces de red de los Pods del 5GC y de la RAN levantados en el clúster.

Puertos relevantes para rfsim

```
===== pod/oai-gnb-86dd67f548-jm6zf =====
Netid   State    Recv-Q   Send-Q     Local Address:Port     Peer Address:Port   Process
udp     UNCONN   0        0            172.21.8.91:2152          0.0.0.0:*       users:(("nr-softmodem",pid=1,fd=100))
tcp     LISTEN   0        5              0.0.0.0:4043            0.0.0.0:*       users:(("nr-softmodem",pid=1,fd=102))
tcp     ESTAB    307740   0            172.21.6.94:4043      172.21.6.27:56582   users:(("nr-softmodem",pid=1,fd=104))
===== pod/oai-nr-ue-668664b8c-g7fpc =====
Netid  State  Recv-Q  Send-Q   Local Address:Port    Peer Address:Port Process
tcp    ESTAB  491616  0          172.21.6.27:56582    172.21.6.94:4043  users:(("nr-uesoftmodem",pid=1,fd=100))
```

Figura A.2. Puertos y conexiones relevantes para el enlace rfsimulator en los Pods del gNB y del nr-UE.

Estado de los Pods

```
NAME                              READY   STATUS    RESTARTS       AGE     IP             NODE       NOMINATED NODE   READINESS GATES
basic-mysql-7c586fd5b8-4l28w      1/1     Running   0              3d21h   10.244.0.155   minikube   <none>           <none>
oai-amf-56946b77bc-c8dbl          2/2     Running   0              3d3h    10.244.0.187   minikube   <none>           <none>
oai-ausf-6bc47c8f84-2k5c2         2/2     Running   0              3d21h   10.244.0.154   minikube   <none>           <none>
oai-gnb-86dd67f548-jm6zf          1/1     Running   0              3d3h    10.244.0.188   minikube   <none>           <none>
oai-nr-ue-668664b8c-g7fpc         1/1     Running   41 (10m ago)   3d3h    10.244.0.190   minikube   <none>           <none>
oai-nrf-fdcd9dfc4-sxt4v           2/2     Running   0              3d21h   10.244.0.159   minikube   <none>           <none>
oai-smf-5c9fbbd549-h72lh          2/2     Running   0              3d3h    10.244.0.191   minikube   <none>           <none>
oai-udm-74978686bc-fqcbm          2/2     Running   0              3d21h   10.244.0.160   minikube   <none>           <none>
oai-udr-68d69698db-zm49s          2/2     Running   0              3d21h   10.244.0.162   minikube   <none>           <none>
oai-upf-76f67878f6-7cmk5          2/2     Running   0              3d17h   10.244.0.170   minikube   <none>           <none>
```

Figura A.3. Estado de los Pods del despliegue.

Estado de los Deployments

```
NAME         READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS    IMAGES                                                                                                SELECTOR
basic-mysql  1/1     1            1           16d   basic-mysql   docker.io/mysql:5.7                                                                                   app.kubernetes.io/instance=basic,app.kubernetes.io/name=basic-mysql
oai-amf      1/1     1            1           16d   tcpdump,amf   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-amf:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-amf
oai-ausf     1/1     1            1           16d   tcpdump,ausf  docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-ausf:v2.0.1                          app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-ausf
oai-gnb      1/1     1            1           6d8h  gnb           docker.io/oaisoftwarealliance/oai-gnb:2023.w49                                                        app.kubernetes.io/instance=gnb,app.kubernetes.io/name=oai-gnb
oai-nr-ue    1/1     1            1           6d8h  nr-ue         docker.io/oaisoftwarealliance/oai-nr-ue:2023.w49                                                      app.kubernetes.io/instance=ue,app.kubernetes.io/name=oai-nr-ue
oai-nrf      1/1     1            1           16d   tcpdump,nrf   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-nrf:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-nrf
oai-smf      1/1     1            1           16d   tcpdump,smf   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-smf:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-smf
oai-udm      1/1     1            1           16d   tcpdump,udm   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-udm:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-udm
oai-udr      1/1     1            1           16d   tcpdump,udr   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-udr:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-udr
oai-upf      1/1     1            1           16d   tcpdump,upf   docker.io/corfr/tcpdump:latest,docker.io/oaisoftwarealliance/oai-upf:v2.0.1                           app.kubernetes.io/instance=basic,app.kubernetes.io/name=oai-upf
```

Figura A.4. Estado de los Deployments del despliegue.

Estado de los Persistent Volume Claims (PVC)

```
NAME       STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
cn5g-pvc   Bound    pvc-d886d63b-c8ef-4218-af79-2f6487b5c1f4   5Gi        RWO            standard       <unset>                 16d
```

Figura A.5. Estado de los PersistentVolumeClaims (PVC).

Ejemplo de Logs NR-UE funcional

```
[NR_PHY]   ============================================
[NR_PHY]   Harq round stats for Downlink: 28513/1/0
[NR_PHY]   ============================================
[NR_PHY]   ============================================
[NR_PHY]   Harq round stats for Downlink: 28519/1/0
[NR_PHY]   ============================================
[NR_PHY]   ============================================
[NR_PHY]   Harq round stats for Downlink: 28526/1/0
[NR_PHY]   ============================================
[NR_PHY]   ============================================
[NR_PHY]   Harq round stats for Downlink: 28532/1/0
[NR_PHY]   ============================================
[NR_PHY]   ============================================
[NR_PHY]   Harq round stats for Downlink: 28538/1/0
[NR_PHY]   ============================================
```

Figura A.6. Extracto de los registros del nr-UE en funcionamiento.

Ejemplo de Logs gNB funcional

```
[NR_MAC]   Frame.Slot 256.0
UE RNTI 8083 (1) PH 0 dB PCMAX 0 dBm, average RSRP -44 (16 meas)
UE 8083: dlsch_rounds 30796/1/0/0, dlsch_errors 0, pucch0_DTX 0, BLER 0.00000 MCS 9
UE 8083: dlsch_total_bytes 3788116
UE 8083: ulsch_rounds 30800/0/0/0, ulsch_DTX 0, ulsch_errors 0, BLER 0.00000 MCS 9
UE 8083: ulsch_total_bytes_scheduled 3572708, ulsch_total_bytes_received 3572708
UE 8083: LCID 1: TX            864 RX            282 bytes
UE 8083: LCID 2: TX              0 RX              0 bytes
UE 8083: LCID 4: TX             21 RX            378 bytes

[NR_MAC]   Frame.Slot 384.0
UE RNTI 8083 (1) PH 0 dB PCMAX 0 dBm, average RSRP -44 (16 meas)
UE 8083: dlsch_rounds 30808/1/0/0, dlsch_errors 0, pucch0_DTX 0, BLER 0.00000 MCS 9
UE 8083: dlsch_total_bytes 3789592
UE 8083: ulsch_rounds 30812/0/0/0, ulsch_DTX 0, ulsch_errors 0, BLER 0.00000 MCS 9
UE 8083: ulsch_total_bytes_scheduled 3574100, ulsch_total_bytes_received 3574100
UE 8083: LCID 1: TX            864 RX            282 bytes
UE 8083: LCID 2: TX              0 RX              0 bytes
UE 8083: LCID 4: TX             21 RX            378 bytes

[NR_MAC]   Frame.Slot 512.0
UE RNTI 8083 (1) PH 0 dB PCMAX 0 dBm, average RSRP -44 (16 meas)
UE 8083: dlsch_rounds 30821/1/0/0, dlsch_errors 0, pucch0_DTX 0, BLER 0.00000 MCS 9
UE 8083: dlsch_total_bytes 3791191
UE 8083: ulsch_rounds 30825/0/0/0, ulsch_DTX 0, ulsch_errors 0, BLER 0.00000 MCS 9
UE 8083: ulsch_total_bytes_scheduled 3575608, ulsch_total_bytes_received 3575608
UE 8083: LCID 1: TX            864 RX            282 bytes
UE 8083: LCID 2: TX              0 RX              0 bytes
UE 8083: LCID 4: TX             21 RX            378 bytes
```

Figura A.7. Extracto de los registros del gNB en funcionamiento.

Ejemplo de Logs AMF funcional

```
[2026-09-01 16:09:54.585] [amf_app] [info] |------------------------------------------------------------------------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info] |-----------------------------------------------gNBs' information-----------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info] |    Index    |      Status      |       Global ID       |      gNB Name       |              PLMN              |
[2026-09-01 16:09:54.585] [amf_app] [info] |      1      |    Connected     |        0xe000         |      FGE-USTHB      |               001, 01          |
[2026-09-01 16:09:54.585] [amf_app] [info] |------------------------------------------------------------------------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info]
[2026-09-01 16:09:54.585] [amf_app] [info] |------------------------------------------------------------------------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info] |-------------------------------------------------UEs' information-------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info] | Index |      5GMM state      |      IMSI       |     GUTI      | RAN UE NGAP ID | AMF UE ID |  PLMN   |  Cell ID  |
[2026-09-01 16:09:54.585] [amf_app] [info] |      1|       5GMM-REGISTERED|   001010000000100|               |               1|         43| 001, 01 |0x   e00000|
[2026-09-01 16:09:54.585] [amf_app] [info] |------------------------------------------------------------------------------------------------------------|
[2026-09-01 16:09:54.585] [amf_app] [info]
[2026-09-01 16:09:56.365] [amf_sbi] [info] Receive Update NF Instance Request, handling ...
[2026-09-01 16:09:56.365] [amf_sbi] [info] Send HTTP message to http://oai-nrf:80/nnrf-nfm/v1/nf-instances/2aca7722-26c7-4250-b320-2b6ea652e4ad
[2026-09-01 16:09:56.365] [amf_sbi] [info] HTTP message Body: [{"op":"replace","path":"/nfStatus","value":"REGISTERED"}]
[2026-09-01 16:09:56.366] [amf_sbi] [info] Get response with HTTP code (204)
[2026-09-01 16:09:56.366] [amf_sbi] [info] Could not get JSON content from the response
[2026-09-01 16:09:56.366] [amf_sbi] [info] Get response with Json content: null
[2026-09-01 16:10:06.366] [amf_sbi] [info] Receive Update NF Instance Request, handling ...
[2026-09-01 16:10:06.366] [amf_sbi] [info] Send HTTP message to http://oai-nrf:80/nnrf-nfm/v1/nf-instances/2aca7722-26c7-4250-b320-2b6ea652e4ad
[2026-09-01 16:10:06.366] [amf_sbi] [info] HTTP message Body: [{"op":"replace","path":"/nfStatus","value":"REGISTERED"}]
[2026-09-01 16:10:06.368] [amf_sbi] [info] Get response with HTTP code (204)
[2026-09-01 16:10:06.368] [amf_sbi] [info] Could not get JSON content from the response
[2026-09-01 16:10:06.368] [amf_sbi] [info] Get response with Json content: null
```

Figura A.8. Extracto de los registros del AMF en funcionamiento.

Ejemplo de Logs AUSF funcional

```
[2026-09-01 16:11:54.986] [ausf_nrf] [info] Sending NF heartbeat request
[2026-09-01 16:11:54.987] [ausf_app] [info] Send HTTP message with body [{"op":"replace","path":"/nfStatus","value":"REGISTERED"}]
[2026-09-01 16:11:54.988] [ausf_app] [info] Get response with HTTP code (204)
```

Figura A.9. Extracto de los registros del AUSF en funcionamiento.

Ejemplo de Logs NRF funcional

```
[2026-09-01 16:13:35.065] [nrf_sbi] [info] Got a request to update an NF instance, Instance ID: d1cec161-4d57-4ee5-8653-c932eb2fd067
[2026-09-01 16:13:35.065] [nrf_app] [info] Handle Update NF Instance request (HTTP version 2)
[2026-09-01 16:13:35.065] [nrf_app] [info] NF Heart-Beat procedure!
[2026-09-01 16:13:35.065] [nrf_app] [info] Updated the NF profile (profile ID d1cec161-4d57-4ee5-8653-c932eb2fd067)
[2026-09-01 16:13:36.347] [nrf_sbi] [info]
[2026-09-01 16:13:36.347] [nrf_sbi] [info] Got a request to update an NF instance, Instance ID: 75e0f309-9c8f-40f0-8b74-08f50687e0e3
[2026-09-01 16:13:36.347] [nrf_app] [info] Handle Update NF Instance request (HTTP version 2)
[2026-09-01 16:13:36.347] [nrf_app] [info] NF Heart-Beat procedure!
[2026-09-01 16:13:36.347] [nrf_app] [info] Updated the NF profile (profile ID 75e0f309-9c8f-40f0-8b74-08f50687e0e3)
[2026-09-01 16:13:36.416] [nrf_sbi] [info]
[2026-09-01 16:13:36.416] [nrf_sbi] [info] Got a request to update an NF instance, Instance ID: 2aca7722-26c7-4250-b320-2b6ea652e4ad
[2026-09-01 16:13:36.416] [nrf_app] [info] Handle Update NF Instance request (HTTP version 2)
[2026-09-01 16:13:36.416] [nrf_app] [info] NF Heart-Beat procedure!
[2026-09-01 16:13:36.416] [nrf_app] [info] Updated the NF profile (profile ID 2aca7722-26c7-4250-b320-2b6ea652e4ad)
[2026-09-01 16:13:37.116] [nrf_app] [info]
Handle heartbeat timeout (NF update), NF instance ID 75e0f309-9c8f-40f0-8b74-08f50687e0e3, time 1788279142753, current ms 1788279217116
[2026-09-01 16:13:37.762] [nrf_sbi] [info]
[2026-09-01 16:13:37.762] [nrf_sbi] [info] Got a request to update an NF instance, Instance ID: 51bc904c-4701-4940-a461-f7272316d4c2
[2026-09-01 16:13:37.762] [nrf_app] [info] Handle Update NF Instance request (HTTP version 2)
[2026-09-01 16:13:37.762] [nrf_app] [info] NF Heart-Beat procedure!
[2026-09-01 16:13:37.762] [nrf_app] [info] Updated the NF profile (profile ID 51bc904c-4701-4940-a461-f7272316d4c2)
[2026-09-01 16:13:39.769] [nrf_sbi] [info]
[2026-09-01 16:13:39.769] [nrf_sbi] [info] Got a request to update an NF instance, Instance ID: 1d9f34b2-de31-4b55-9789-854458595660
[2026-09-01 16:13:39.769] [nrf_app] [info] Handle Update NF Instance request (HTTP version 2)
[2026-09-01 16:13:39.769] [nrf_app] [info] NF Heart-Beat procedure!
[2026-09-01 16:13:39.769] [nrf_app] [info] Updated the NF profile (profile ID 1d9f34b2-de31-4b55-9789-854458595660)
```

Figura A.10. Extracto de los registros del NRF en funcionamiento.

Ejemplo de Logs SMF funcional

```
[2026-09-01 15:52:26.123] [smf_app] [info] Find PDU Session with ID 10
[2026-09-01 15:52:26.123] [smf_app] [info] Sending ITTI message 36itti_n4_session_modification_request to task TASK_SMF_N4
[2026-09-01 15:52:26.124] [smf_n4 ] [info] handle_receive(31 bytes)
[2026-09-01 15:52:26.124] [smf_app] [info] Handle N4 Session Modification Response (PDU Session Id 10)
[2026-09-01 15:52:26.124] [smf_app] [warning] Could not get DL FTEID from PDR in DL
[2026-09-01 15:52:26.124] [smf_app] [info] PDU Session Establishment Request (UE-Initiated)
[2026-09-01 15:52:26.124] [smf_app] [info] Set PDU Session Status to PDU_SESSION_ACTIVE
[2026-09-01 15:52:26.124] [smf_app] [info] Set upCnxState to UPCNX_STATE_ACTIVATED
[2026-09-01 15:52:26.124] [smf_app] [info] SMF context:

SMF CONTEXT:
SUPI:                           001010000000100
PDU SESSION:
        PDU Session ID:                 10
        DNN:                    oai
        S-NSSAI:                        SST=1, SD=16777215
        PDN type:               IPV4
        PAA IPv4:               12.1.1.100
        Default QFI:            1
        SEID:                   82
        N3:
                QoS Flow:
                        QFI:            1
                        UL FTEID:       TEID=41, IPv4=172.21.8.95
                        DL FTEID:       TEID=1619213485, IPv4=172.21.8.91
                        PDR ID UL:      1
                        PDR ID DL:      2
                        Precedence:     0
                        FAR ID UL:      1
                        FAR ID DL:      2
```

Figura A.11. Extracto de los registros del SMF en funcionamiento.

uc3m | Universidad Carlos III de Madrid

Anexo 2 – DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE GRADO (TFG)

He usado IAG en mi TFG

Marca lo que corresponda:

SI	**NO**

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFG puede verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

Pregunta	
1. En mi interacción con herramientas de IAG he facilitado **datos de carácter confidencial** contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).	
SÍ, he usado estos datos con la autorización de los interesados	NO, no he usado datos de carácter confidencial
2. En mi interacción con herramientas de IAG he facilitado **materiales protegidos por derechos de autoría** contando siempre con la autorización de los respectivos titulares.	
SÍ, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley:	NO, no he usado materiales protegidos por derechos de autoría

• obra en dominio público • obra licenciada (licencias Creative Commons) • uso de fragmentos con fines de investigación (derecho de cita)	
3. En mi interacción con herramientas de IAG he facilitado **datos de carácter personal** con la debida autorización de los interesados.	
SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	NO, no he usado datos de carácter personal
4. Mi utilización de la herramienta de IAG ha **respetado sus términos de uso**, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
SI	NO

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IAG. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo].*

Declaro haber hecho uso del sistema de IAG (Nombre del sistema/herramienta IAG y versión: ChatGPT, Gemini, Copilot…) **para:**

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG*

Declaro haber usado Gemini Opus5 para: Asesorarme al inicio del proyecto sobre qué tecnología utilizar para desplegar el clúster de Kubernetes. Me sirvió de guía para comparar opciones como Minikube y K3s, ya que no tenía experiencia previa en entornos mono-nodo.

- *Revisión o reescritura de párrafos redactados previamente*

Declaro haber usado Gemini Opus5 para: Revisar la ortografía y mejorar la redacción de algunos párrafos hacia un estilo más formal. También me apoyé en la IA para gestionar las alusiones a otras partes del documento, logrando así reducir las repeticiones de contenido.

- *Búsqueda de información o respuesta a preguntas concretas*

Declaro haber usado Gemini Opus5 para: Ayudarme a encontrar documentación sobre los problemas detectados. Su uso fue clave para buscar información dentro de los estándares 3GPP, ya que su gran volumen y compleja estructura resultan confusos para alguien que los consulta por primera vez.

- *Búsqueda de bibliografía*

Declaro haber usado Gemini Opus5 para: Ayudarme a navegar por 3GPP para la bibliografía. Y en la bibliografía del ámbito regulador.

- *Resumen de bibliografía consultada*

NA: No se ha utilizado IAG para este fin.

- *Traducción de texto consultados*

NA: No se ha utilizado IAG para este fin.

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

Declaro el uso de Gemini Opus5 para la optimización en la búsqueda de comandos de Linux y Kubernetes, agilizando el flujo de trabajo en la terminal. Por el contrario, se descartó su empleo para el diagnóstico de errores en las plantillas YAML de Helm; la experiencia demostró que las sugerencias de la herramienta introducían anomalías adicionales en lugar de resolver los fallos existentes, por lo que esta tarea pasó a gestionarse de forma manual.

- *Generación de esquemas, imágenes, audios o vídeos*

Declaro el uso de Gemini Opus5 en las fases iniciales de diseño para la generación preliminar de esquemas de la red 5GSA. No obstante, debido a las limitaciones de la herramienta para representar con precisión la arquitectura técnica requerida, dichos modelos se emplearon únicamente como elementos de reserva (placeholders) y no se incluyeron en la versión definitiva.

- *Procesos de optimización*

NA: No se ha utilizado IAG para este fin.

- *Tratamiento de datos: recogida, análisis, cruce de datos…*

Declaro el uso de la herramienta Gemini Opus5 para el análisis y filtrado de los registros (logs) del sistema. Su empleo facilitó la identificación de información crítica frente al elevado volumen de datos concurrentes generado por los pods, optimizando así el proceso de depuración técnica.

- *Inspiración de ideas en el proceso creativo*

NA: No se ha utilizado IAG para este fin.

- *Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo*

Si crees necesario incluirlo, añade, en cada uno de los casos:

empleando el *prompt* …
(escribe la petición realizada a la IAG)
teniendo como interacción …
(describe qué interacción realizaste con la IAG tras la respuesta del prompt)

Ejemplo: ***Declaro*** *haber hecho uso del sistema de IAG* ***Chat GPT X.x*** *para* ***buscar información*** *empleando* ***el prompt: "Dime un ejemplo concreto que ilustre la aplicación de la propuesta***

urbanística de la Ciudad de los 15 minutos en España" *teniendo como interacción* ***la proposición del ejemplo concreto del distrito de Chamartín que se ha reflejado en la memoria***.

Parte 3: reflexión sobre utilidad

Aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IAG en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

Como principal debilidad, la herramienta demostró ser un recurso de doble filo en la fase de desarrollo, induciendo en ocasiones a bucles de resolución ineficaces. Un ejemplo crítico de esto ocurrió tras levantar con éxito todas las interfaces de red a excepción de la N6 (salida a internet); al aplicar una configuración específica recomendada por la IAG para corregir un registro de error (log), el sistema experimentó una caída total del resto de las interfaces, lo que requirió dos horas de diagnóstico técnico y reversión manual por mi parte debido a un exceso de confianza inicial en la herramienta.

En contraposición, entre sus mayores fortalezas, la IAG ha sido un motor clave en mi proceso de aprendizaje, permitiéndome asimilar conceptos y localizar recursos de documentación con gran rapidez. Asimismo, su valor en la etapa final de desarrollo y en la extracción de conclusiones resultó de extrema utilidad, actuando como un asistente eficiente para el filtrado de datos masivos y la depuración de la memoria.

En conclusión, se valora la IAG como una herramienta complementaria muy potente, siempre que se asuma de forma crítica que sus propuestas técnicas requieren una supervisión y validación humana estrictas antes de ser implementadas.